

基于图模型与传播仿真的校园社交网络分析

摘 要

本文针对校园社交平台“好友关系与社交价值”分析问题，基于 200 名用户、1198 条好友关系以及用户属性与行为数据，构建校园社交网络图模型。围绕社群识别、好友推荐、信息传播和定向推送优化四个任务展开研究，旨在为平台的关系挖掘与运营决策提供可解释、可验证的量化依据。

针对问题一，本文将好友关系抽象为无向图，采用 Louvain 社群发现算法识别主要社群结构，并通过内部密度、跨社群连接密度和功能重叠指数刻画社群差异与相互关系。结果表明，**样本网络可稳定划分为 10 个社群，模块度为 0.247199**，其中 C_4 在多个高密度社群之间承担桥梁作用，前 5 个高密度社群在成员归属上不重叠，但在兴趣组织和年级结构层面存在明显功能交叉。

针对问题二，本文将面向 S11 的好友推荐建模为可解释链路预测问题，构建融合结构、属性与行为信息的 SAB 综合评分模型。模型在保留共同好友、Jaccard 系数和 Adamic-Adar 指数等结构特征的同时，引入专业背景、社团参与、活跃时段和话题偏好等画像信息。结果表明，**S146、S24 和 S2 为最优推荐对象**，三者分别体现出画像匹配度高、兴趣组织联系强和结构连接强这三类推荐情况。

针对问题三，本文围绕“科技类话题”建立关键用户综合得分模型，并结合题目给出的转发概率公式进行离散时间 Monte Carlo 传播仿真。模型综合考虑度中心性、介数中心性、接近中心性、 k -core 指标、互动频率、发帖频率、科技话题参与度以及好友首轮可见速度，先采用等权方案形成预评分，再用传播结果反向修正排序。结果表明，**S6 为科技类话题最优传播种子**，其在 48 小时内的平均覆盖人数达到 199.13，且在权重替换和概率参数扰动下结果较稳定。

针对问题四，本文将文化类话题的定向推送建模为带阶段配额约束的传播优化问题，采用“高相关初始解 + 阶段贪心 + 局部交换修正”的求解策略，并与纯贪心、最高度数、最高活跃度和最高文化参与度等基线方案进行比较。结果表明，**本文优化策略在 48 小时内实现平均 199.80 人的传播覆盖**，优于所有对比基线，说明在主题相关用户中补入跨社群用户，有助于提高早期扩散效率。

模型检验结果表明，本文各子模型在算法替换、特征消融和参数扰动条件下均具有较好的稳定性。全文优先采用可解释图模型，并结合仿真检验，所提出的运营建议可为同类校园社交平台的关系挖掘与推送决策提供参考。

关键词：校园社交网络 社群发现 好友推荐 信息传播
Louvain 算法 Monte Carlo 仿真

目 录

1	问题重述	4
2	问题分析	4
2.1	数据结构分析	4
2.2	分问建模拆解	4
2.3	总体思路	5
3	模型假设与符号说明	6
3.1	模型假设	6
3.2	主要符号与指标说明	7
4	数据预处理与描述性统计	8
4.1	预处理步骤	8
4.2	网络基础统计	8
4.3	属性分布统计	9
4.4	行为数据统计	10
4.5	网络结构补充图示	10
5	第 1 问：社群识别与社群关系分析	11
5.1	建模思路	11
5.2	社群关系刻画指标	12
5.3	求解流程与参数设置	13
5.4	主社群识别结果	13
5.5	社群关系分析	16
5.6	功能重叠分析	16
5.7	本问结论	18
6	第 2 问：面向 S11 的好友推荐模型	19
6.1	建模思路	19
6.2	候选集构建	19
6.3	特征设计与归一化	19
6.4	SAB 综合评分模型	20
6.5	求解流程与参数设置	21
6.6	推荐结果	22
6.7	推荐结果分析	22
6.8	本问结论	23
7	第 3 问：关键用户筛选与传播仿真	24
7.1	建模思路	24
7.2	候选用户筛选	24

7.3	候选用户预评分	24
7.4	传播机制与 Monte Carlo 仿真	26
7.5	联合求解流程与参数设置	27
7.6	关键用户排序结果	28
7.7	48 小时传播结果	29
7.8	本问结论	30
8	第 4 问：定向推送配额约束下的传播优化	30
8.1	建模思路	30
8.2	候选池构建与阶段划分	31
8.3	优化模型	31
8.4	求解流程与参数设置	32
8.5	策略比较结果	34
8.6	优化策略结构分析	35
8.7	本问结论	35
9	模型检验与敏感性分析	35
9.1	第 1 问模型检验	36
9.2	第 2 问模型检验	37
9.3	第 3 问模型检验	39
9.4	第 4 问模型检验	40
10	模型评价、改进与推广	40
10.1	模型优点	40
10.2	模型缺点	41
10.3	模型改进	41
10.4	模型推广	41
10.5	主要结论	41
10.6	平台建议	42
	附录	44
	附录 1 支撑材料与结果文件列表	44
	附录 2 程序文件说明	45
	附录 3 四问模型总览流程图	46
	附录 4 第 4 问两阶段推送优化的详细求解过程	49
	附录 5 详细结果表	49
	附录 6 第 1 问核心算法代码	52
	附录 7 第 2 问核心算法代码	53
	附录 8 第 3 问核心算法代码	58
	附录 9 第 4 问核心算法代码	63

一、 问题重述

随着移动互联网深入校园生活，校园社交平台已不只用于日常交流，还涉及课程资料共享、社团组织、竞赛组队、信息传播与公共服务触达等任务。题目所研究的平台，已经成为连接学习、生活与校园治理的线上协同空间，因此其中的好友关系、用户属性与行为数据，不仅反映学生之间的社交联系，也反映信息在不同群体之间的流动路径。

然而，平台运营往往面临活跃度向少数用户集中、不同院系和社团之间传播效率不均、推荐结果容易停留在熟人圈层等问题。如何识别主要社群、挖掘潜在好友、找到更适合启动传播的用户，并在有限推送配额下提升内容触达效果，正是题目要解决的问题。结合题意，本文需要完成以下四个任务：

1. 基于好友关系表与用户属性表识别平台中的主要社群，分析社群之间的关系强弱，并重点给出连接密度最大的 5 个社群及其功能特征。
2. 根据网络结构、用户属性与行为特征，为用户 s11 从非好友用户中筛选出最适合推荐的 3 位候选对象，并解释推荐原因及尚未成为好友的可能原因。
3. 在给定传播规则下，筛选有利于科技类话题扩散的关键用户，并模拟该用户于中午 12:00 发帖后 48 小时内的信息传播过程。
4. 在定向推送配额受限的条件下，设计文化类话题的推送优化方案，并比较不同策略下的传播效果。

二、 问题分析

2.1 数据结构分析

题目提供的原始数据包括三张核心表：

- 好友关系表：描述用户之间的好友连接关系，可抽象为无向图中的边。
- 用户属性表：给出年级、专业类别、具体专业、所属社团和班级等静态属性，可用于解释社群来源和推荐相似性。
- 行为数据表：包含发帖数量、平均互动频率、活动参与次数、科技类与文化类话题参与度以及活跃时段等动态特征，可用于推荐与传播分析。

2.2 分问建模拆解

1. **第 1 问：围绕社群划分、社群关系和功能重叠展开。**题目首先给出的是好友关系表，因此最直接的数据对象是无向社交网络；而题目要求输出的不只是“分成多少群”，还包括“连接最紧密的 5 个社群”“社群之间关系强弱”“是否存在重叠”。这说明第 1 问不能停留在单纯聚类，而要继续完成社群画像统计与社群间比较。基于这一拆解，本文先用社群发现算法给出主划分，再用内部密度刻画社

群紧密程度，用跨社群连接密度刻画社群间联系强弱，最后借助年级、专业和社团分布衡量功能重叠。之所以采用 Louvain 模型，是因为其直接对应网络社群划分问题，且模块度优化结果便于解释和后续比较。

2. **第 2 问：围绕候选筛选、相似性度量和推荐排序展开。**题目要求从 S11 当前尚未建立好友关系的用户中选出 3 位最合适的推荐对象，并解释推荐原因及尚未成为好友的可能原因。因此，本问不仅要判断“谁更可能成为好友”，还要说明“为什么推荐”。从网络角度看，可以借鉴链路预测思想，即利用已有好友关系判断潜在好友连接的可能性。经典链路预测方法中，共同好友数、Jaccard 系数和 Adamic-Adar 指数等结构指标能够刻画两个用户在网络中的接近程度；但本题还提供了年级、专业、班级、社团、活跃时段和话题偏好等属性与行为信息，仅依靠结构指标无法充分解释好友形成原因。基于此，本文在结构相似性指标的基础上，引入属性相似性和行为相似性，构建结构-属性-行为融合的 SAB 综合评分模型，对 S11 的非好友候选用户进行可解释排序。
3. **第 3 问：围绕关键用户筛选与传播模拟展开。**题目并不是只问“谁最重要”，而是要求在科技类话题下筛选关键用户，并模拟其在正午发帖后 48 小时内的传播过程。因此，这一问包含两个子任务：第一，依据网络位置、互动强度、主题相关性和好友时序匹配度筛选候选种子；第二，依据题面给出的转发概率公式和活跃时段规则，模拟内容沿好友网络的离散传播。也就是说，第 3 问必须同时包含静态评分模型和动态传播模型，前者负责缩小候选范围，后者负责验证扩散效果，这也是本文将综合得分与 Monte Carlo 仿真结合起来的原因。
4. **第 4 问：围绕候选池筛选、阶段决策与组合优化展开。**选做题不再是找单个关键用户，而是要在文化类话题场景下，面对每天仅有 10 个推送名额的约束，决定两天内共 20 个推送对象的选择顺序与组合。由于被推送用户会立即看到内容，这一问可以看作带阶段配额约束的影响力最大化问题。它既要考虑用户对文化话题的相关性，也要考虑跨社群连接作用与时段匹配带来的边际收益，还要处理“先推谁、后推谁”的阶段决策。基于这一问题结构，本文先筛选活跃且对文化类内容敏感的候选池，再用阶段贪心构造初始方案，最后用局部交换搜索提升组合质量。

2.3 总体思路

由于本题更强调建模合理性、结果正确性和文字表达清晰度，因此本文优先采用可解释的图模型和加权评分模型，再通过稳健性分析、传播仿真和受限优化增强结果可信度。整体上，本文按社群结构识别、好友推荐、关键用户扩散和定向推送优化四个层次依次展开。

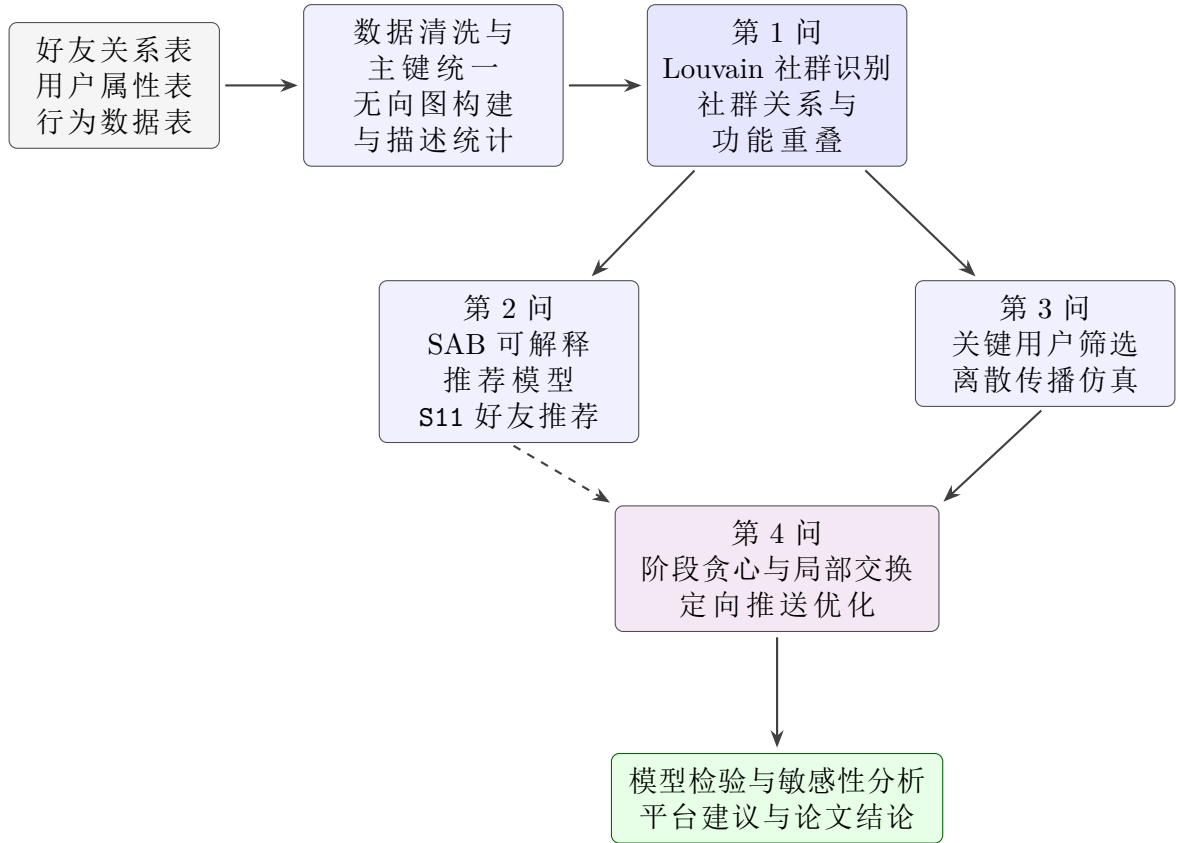


图 1: 本文从原始数据到四问求解与模型检验的整体建模流程

图 1 给出了本文从数据整理到模型检验的整体求解过程。其中，第 1 问给出网络结构基础，第 3 问的传播能力评估支撑第 4 问的受限优化，模型检验则贯穿全文。

三、 模型假设与符号说明

3.1 模型假设

为便于建立模型，作如下基本假设：

1. 由于好友关系表仅记录是否存在好友关系，未给出关系强度，本文将所有好友边视为等权边，并依据好友关系的对称性将其抽象为无向图。
2. 题目给出的用户属性在 48 小时研究窗口内保持不变。
3. 好友关系在研究周期内保持静态，暂不考虑用户新增或删除好友边的动态变化。
4. 用户活跃时段可以代表其主要上线窗口，用户在对应时段更可能接触并传播信息。
5. 在第 1 问的社群识别阶段，本文采用标准 Louvain 算法进行非重叠主社群划分，每个用户仅被赋予一个主社群标签。
6. 在第 1 问中，本文以年级、专业和社团分布刻画社群功能定位，并在功能重叠度量中对三个维度作等权处理。

7. 同类话题下，用户转发行为仅由题目给定的转发概率公式决定，不受具体信息内容差异的影响。
8. 推送用户与普通发帖用户的传播规则一致，仅在初始触达方式上存在区别。
9. 传播仿真中的随机事件采用多次重复实验，以均值结果反映总体传播水平。

3.2 主要符号与指标说明

为便于阅读，本文将后续四问中重复出现的核心符号与指标统一整理如表 1 所示。

表 1: 主要符号与指标总表

模块	符号	含义
基础网络符号		
基础	$G = (V, E)$	校园社交网络图，其中 V 为用户节点集合， E 为好友关系边集合
基础	$N(u)$	用户 u 的好友邻居集合
基础	A_{ij}	邻接矩阵元素，表示节点 i, j 是否存在好友边
基础	k_i	节点 i 的度数，即其直接好友数量
基础	m	全网边数
第 1 问：社群识别		
第 1 问	C_k	第 k 个社群
第 1 问	n_k, m_k	社群 C_k 的节点数与内部边数
第 1 问	ρ_k	社群 C_k 的内部密度
第 1 问	e_{ab}	社群 C_a 与 C_b 之间的跨社群边数
第 1 问	η_{ab}	社群 C_a 与 C_b 之间的跨社群连接密度
第 1 问	ω_{ab}	社群 C_a 与 C_b 的功能重叠指数
第 2 问：好友推荐		
第 2 问	Ω	面向 S11 的候选推荐用户集合
第 2 问	$x_i(\text{S11}, v)$	用户 v 相对于 S11 的第 i 个归一化特征值
第 2 问	$\tau_{\text{S11}, v}$	S11 与候选用户 v 的话题偏好相似度
第 2 问	$s_{\text{S11}, v}^{\text{SAB}}$	SAB 模型下 S11 与候选用户 v 的综合推荐得分
第 3 问：关键用户与传播		
第 3 问	Ω_3	科技类话题关键用户候选池
第 3 问	$\tilde{A}_u$	归一化后的科技类话题参与度（用于预评分）
第 3 问	$\tilde{F}_u$	归一化后的平均互动频率（用于预评分）

续下页

表 1（续）主要符号与指标总表

模块	符号	含义
第 3 问	A_u	科技类话题参与度原始值（用于转发概率公式）
第 3 问	F_u	平均互动频率原始值（用于转发概率公式）
第 3 问	S_u	用户 u 的关键用户预评分
第 3 问	Q_u	用户 u 的好友首轮可见速度得分
第 3 问	$\tau(v, t)$	当某次曝光发生于时刻 t 时，用户 v 首次看到帖子的时刻
第 3 问	$P_{\text{share}}(u, \Delta t_u)$	用户 u 在给定延迟下的转发概率
第 3 问	M_u	用户 u 在传播仿真中的综合传播表现分值
第 3 问	Z_u	用户 u 的最终关键用户得分
第 4 问：推送优化		
第 4 问	$\mathcal{S}$	阶段化定向推送策略集合
第 4 问	$F(\mathcal{S})$	策略 $\mathcal{S}$ 在 48 小时内的期望传播覆盖收益
第 4 问	$\Delta(u \mathcal{S})$	在当前策略 $\mathcal{S}$ 下加入用户 u 带来的边际传播增益
第 4 问	$\phi(u)$	用户 u 在推送优化中的预评分

四、数据预处理与描述性统计

4.1 预处理步骤

对原始数据进行如下处理：

1. 将三张表中的用户编号统一为以学生 ID 为主键的格式。
2. 读取好友关系表，删除自环与重复边，将好友关系视为无向边。
3. 将用户属性表中的社团字段按分隔符拆分，便于统计共同社团数量。
4. 将行为数据表中的话题参与度、互动频率等字段转化为数值变量，便于后续归一化处理。

经核对，用户属性表与行为数据表均覆盖全部 200 个学生 ID，且与建图后的节点集合一致，因此三张表已成功统一到同一主键体系下。以上预处理与后续各问的建模计算均基于 Python 实现，图结构操作与指标计算主要依赖 NetworkX 库，数值统计与结果导出使用 openpyxl 与 Python 标准库完成。

4.2 网络基础统计

根据题目数据初步统计，平台样本网络具有如下特征：

表 2: 网络基础统计结果

指标	数值
用户节点数	200
好友关系边数	1198
原始好友记录数	1198
重边记录数	0
自环记录数	0
孤点个数	0
网络密度	0.0602
平均度	11.98
最大度	59
平均聚类系数	0.1869
连通分量个数	1
最大连通分量规模	200

由表中结果可见，该网络不存在自环、重边与孤点，且整体连通，无需额外连通性修复即可直接作为无向图输入后续的社群发现、链路预测与传播仿真模型。

4.3 属性分布统计

表 3: 年级与活跃时段分布统计

年级	人数	活跃时段	人数
大一	46	上午	16
大二	44	下午	64
大三	44	晚上	102
大四	33	全天	18
研究生	33		

表 4: 专业类别分布统计

专业类别	人数
文科	69
工科	65
理科	66

从属性分布看，样本在年级与专业类别上的分布较为均衡，而活跃时段明显集中于晚上（占比 51%），下午次之（32%），这一特征对信息传播路径和发布时间设置具有直接影响。

4.4 行为数据统计

为进一步了解用户的行为活跃水平与话题偏好分布，对行为数据表中的关键连续变量进行描述性统计，结果如表 5 所示。

表 5: 行为数据描述性统计

指标	均值	中位数	最小值	最大值
发帖数量	11.31	10.0	5	50
平均互动频率	11.50	10.7	4	25
活动参与次数	5.86	5.0	0	20
科技类话题参与度	5.36	5.0	0	10
文化类话题参与度	5.25	5.0	0	10

由表 5 可见，各项行为指标均存在明显的个体差异：发帖数量和活动参与次数的最大值与最小值跨度较大，反映平台用户活跃度差异明显；科技类与文化类话题参与度的中位数均为 5.0 且取值范围为 0 ~ 10，说明两类话题的整体参与水平相当，但存在完全无参与的极端用户。这些特征为后续话题筛选阈值设定和推荐特征归一化提供了数据依据。

4.5 网络结构补充图示

为进一步刻画网络结构特征，绘制度分布图如图 2 所示。

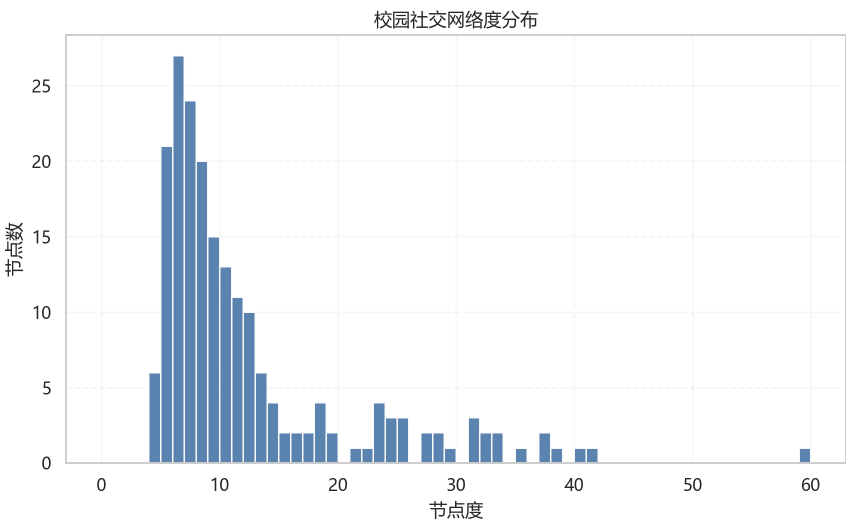


图 2: 校园社交网络度分布

由图 2 可见，大部分节点的度集中在较低区间，但仍存在少量高度节点，说明网络中同时存在普通用户与少数连接能力较强的用户。这一特征为后续社群发现与关键用户识别提供了基础。

五、第 1 问：社群识别与社群关系分析

5.1 建模思路

第 1 问按照“数据输入 → 模型处理 → 输出结果”的思路展开：先以好友关系表构造无向社交网络，并结合用户属性表形成社群画像；再利用 Louvain 算法完成主社群划分，同时计算社群密度、跨社群连接强度与功能重叠指数；最后输出主要社群、前 5 个高密度社群的功能特征，以及社群之间的联系强弱。

在这一框架下，第 1 问的目标是识别校园社交网络中的主要社群，并分析社群之间的联系强弱及前 5 个高密度社群的特征。为此，本文将好友网络抽象为无向图

$$G = (V, E),$$

其中节点表示用户，边表示好友关系。关于复杂网络中的社群发现问题，国内已有针对社区挖掘与在线社交网络群体识别的综述性研究，可为本题的社群解释提供直接参考^[1-3]。国际上则从图划分与模块度优化角度系统总结了社区发现问题，并提出了适用于大规模网络的 Louvain 高效算法^[4,5]。考虑到本题更强调结果可解释性与群体结构刻画，本文采用 Louvain 社群发现算法作为主模型，对网络进行非重叠划分。题目所说的“主要社群”，应当体现为群内连接显著高于随机连接水平，因此本文以模块度衡量某一划分是否较好地联系紧密的用户归到同一社群中。

在当前校园社交网络背景下，模块度衡量的是“社群内部实际好友连接”相对于“在保持每个用户好友数量不变时随机连边所形成的期望内部连接”多出了多少。若某种划分下，同一社群内的实际连边明显多于随机期望，就说明这批用户更接近真实的熟人圈、兴趣圈或协作圈，而不是随机组合的集合。具体地， A_{ij} 表示节点 i, j 之间是否存在实际好友边； $\frac{k_i k_j}{2m}$ 表示在保持节点度数不变时，两节点随机形成连边的期望强度；二者作差，是为了保留“超出随机背景的那部分内部连接”。 $\delta(c_i, c_j)$ 只在节点 i, j 属于同一社群时取 1，因此相当于只统计群内节点对；对全部节点对求和，是为了汇总整张网络的群内超额连接；最后再除以 $2m$ ，一方面是因为无向图在双重求和下会对同一条边计数两次，另一方面也是为了消除网络规模影响，使不同划分结果可以直接比较。于是，模块度定义为

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \quad (1)$$

其中 A_{ij} 为邻接矩阵元素， k_i 为节点 i 的度， m 为网络总边数， $\delta(c_i, c_j)$ 表示节点 i, j 是否属于同一社群。Louvain 算法通过不断寻找能够带来模块度增益的节点移动与社群合并，逐步提高 Q ，从而得到更符合校园社交实际结构的社群划分。

5.2 社群关系刻画指标

社群是否紧密，首先体现在一个圈子内部“已经建立的好友关系”占“理论上可能建立的好友关系”的比例上。若社群 C_k 中有 n_k 个成员，则成员两两之间理论上最多可形成 $\frac{n_k(n_k-1)}{2}$ 条内部边；而实际观测到的内部边只有 m_k 条，因此定义社群 C_k 的内部密度为

$$\rho_k = \frac{2m_k}{n_k(n_k - 1)}, \quad (2)$$

其中 n_k 为社群节点数， m_k 为内部边数。 ρ_k 越大，说明该圈子内部成员彼此认识、互动频繁并形成稳定熟人网络的程度越高；反之则说明该社群更偏向松散聚集。

进一步地，社群之间关系强弱，本质上取决于两个圈子之间“已经发生的跨圈联系”占“理论上可能发生的跨圈联系”的比例。若社群 C_a 与 C_b 的节点数分别为 n_a, n_b ，则两社群之间理论上最多可形成 $n_a n_b$ 条跨社群边；而实际观测到的跨社群边只有 e_{ab} 条，因此定义两个社群 C_a, C_b 之间的跨社群连接密度为

$$\eta_{ab} = \frac{e_{ab}}{n_a n_b}, \quad (3)$$

其中 e_{ab} 为两社群之间的跨社群边数。 η_{ab} 越大，说明两个原本独立的圈子之间存在更多跨越边界的互动、交集或桥梁；反之则说明两个圈子更趋于分离。正是借助 ρ_k 与 η_{ab} ，本文才能分别回答“一个社群内部是否足够紧密”和“两个社群之间是否联系紧密”这两个问题。

对于“是否存在重叠”这一问题，本文不再考察成员交叉，而是从功能层面比较社群画像。由于第 1 问主要基于好友关系和用户属性展开，本文选取年级、专业和社团三个维度构造社群画像：年级反映用户所处培养阶段，专业反映知识背景与学习场景，社团反映兴趣组织与活动连接；三者共同构成校园社交平台中较为稳定的功能属性。相比之下，班级信息粒度更细，更偏向局部行政组织关系；行为数据则更适合放到后续好友推荐和传播分析中使用，因此在第 1 问功能重叠度量中暂不引入。又由于题目没有给出三类属性影响强弱的先验信息，本文在功能重叠指数中对三个维度作等权处理。若两个社群在年级、专业和社团分布上足够接近，即使成员不重合，也可能承担相似功能，因此设社群 C_a, C_b 在这三个维度上的分布向量分别为 $\mathbf{g}_a, \mathbf{g}_b, \mathbf{m}_a, \mathbf{m}_b, \mathbf{h}_a, \mathbf{h}_b$ ，并用余弦相似度刻画对应画像的接近程度：

$$s(\mathbf{x}_a, \mathbf{x}_b) = \frac{\mathbf{x}_a \cdot \mathbf{x}_b}{\|\mathbf{x}_a\|_2 \|\mathbf{x}_b\|_2}. \quad (4)$$

再将三类画像相似度取平均，定义社群 C_a, C_b 的功能重叠指数为

$$\omega_{ab} = \frac{s(\mathbf{g}_a, \mathbf{g}_b) + s(\mathbf{m}_a, \mathbf{m}_b) + s(\mathbf{h}_a, \mathbf{h}_b)}{3}. \quad (5)$$

ω_{ab} 越大，说明两个社群在成员结构和兴趣组织层面越接近。

5.3 求解流程与参数设置

第 1 问的程序求解流程如下。具体实现基于 Python 展开，图结构构建与网络指标计算主要依赖 NetworkX，Louvain 社群发现优先调用 `python-louvain` 包，并固定随机种子为 42 以保证结果可复现：

1. 以去重后的无向好友图作为输入，调用 Louvain 算法进行模块度优化。程序中固定随机种子为 42，首先将每个节点视作独立社群；随后依次扫描各节点，将该节点暂时移动到其相邻节点所在的各个候选社群中，分别计算此次移动带来的模块度增益 ΔQ ，只接受能够使 Q 上升且增益最大的那次移动；当整轮扫描后再无节点迁移能继续提高模块度时，就把当前每个社群压缩为一个“超级节点”，将社群内部边折算为自环权重、社群之间的边折算为超级节点之间的连边，再在压缩后的网络上重复上述“节点移动-社群压缩”过程，直到模块度不再提升。
2. 对得到的每个社群 C_k 统计节点数 n_k 、内部边数 m_k 、内部密度 ρ_k 、平均内部度

$$\bar{d}_k = \frac{2m_k}{n_k},$$

以及对外连边数，同时汇总其年级、专业、班级和社团分布，为后续社群解释提供画像信息。

3. 将全部社群按“内部密度优先、平均内部度次之、规模再次之”的规则排序，选出前 5 个高密度社群。这样的排序方式既强调内部紧密程度，也兼顾社群的组织规模。
4. 对任意两个不同社群进一步计算跨社群边数 e_{ab} 、连接密度 η_{ab} 以及功能重叠指数 ω_{ab} ，分别回答“社群之间关系强弱”和“是否存在重叠”两个子问题。

附录给出了上述过程对应的核心程序，第 1 问的社群划分、排序与画像统计均依照这一流程得到。由此，第 1 问形成了由 Louvain 主社群划分、内部密度排序、跨社群关系度量和功能画像比较组成的建模框架，既能完成社群识别，也能进一步解释社群关系和功能定位差异。

5.4 主社群识别结果

利用 Louvain 算法对整个校园社交网络进行划分，共识别出 10 个社群，所得模块度为 0.247199。这说明网络中已经存在较明显的群体结构，但不同社群之间仍保留了较多跨群连接，符合校园社交网络“既分群又相互联通”的特点。

校园社交网络社群结构图

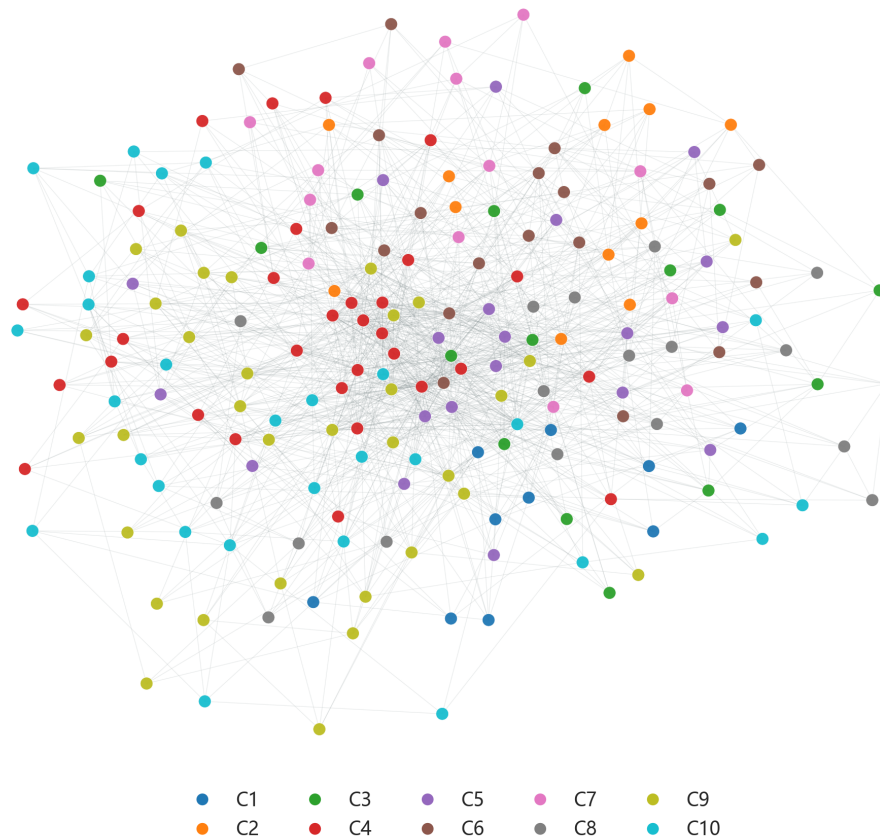


图 3: 基于 Louvain 算法的全网社群划分结果

图 3 直观展示了全网社群划分的整体格局。可以看出，网络中心区域聚集了多个相互交织的社群，而边缘区域则分布着相对稀疏但内部仍保持连接的小团体，这与校园平台中中心活跃群体和外围兴趣圈层并存的组织形态一致。

按照内部密度从高到低排序，前 5 个高密度社群如表 6 所示。

表 6: 前 5 个高密度社群统计结果

社群编号	节点数	内部边数	内部密度	主导属性特征	功能判断
C_1	10	16	0.3556	研究生占比 40%，工科占比 70%，计算机科学占比 60%，学生会最活跃	研究生计算机核心社群
C_2	12	23	0.3485	大四占比 41.67%，理科占比 58.33%，电子科技协会占比 58.33%	高年级理工实践社群
C_3	15	27	0.2571	大一占比 40%，文科占比 80%，法学占比 33.33%，机器人社团占比 40%	大一文科兴趣社群
C_4	31	118	0.2538	工科占比 67.74%，机械工程占比 22.58%，创新创业俱乐部占比 38.71%	工科创新枢纽社群
C_5	20	45	0.2368	大三占比 30%，理科占比 45%，材料科学/生物学较集中，电子科技协会占比 30%	理工混合实践社群

对全部 10 个社群，本文均已统计其规模、内部密度、平均内部度、对外连边数，以及年级、专业、社团分布；考虑正文篇幅，下文重点展示前 5 个高密度社群的结果。

表 6 表明，前 5 个高密度社群并非简单按年级或专业单一聚合，而是同时受到专业背景、兴趣组织和学生活动角色的影响：

1. C_1 的研究生和计算机科学占比明显偏高，说明其更接近学术交流或技术协作型小团体；
2. C_2 和 C_5 均表现出较强的理工实践属性，尤其都与电子科技协会联系紧密；
3. C_3 尽管以文科新生为主，但机器人社团占比较高，说明其兼具兴趣驱动色彩；
4. C_4 规模最大，平均内部度达到 7.6129，说明其不仅内部连接稠密，而且具备较强的组织与辐射能力。

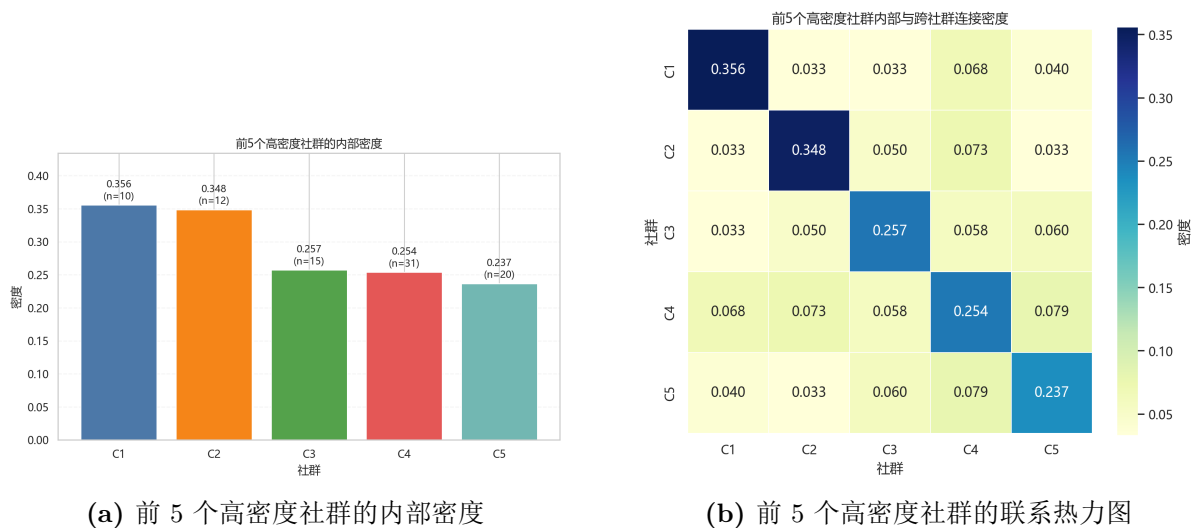


图 4: 前 5 个高密度社群中 C_4 兼具桥梁作用与较高连接强度

图 4 从图形角度验证了表 6 和表 7 的结论：一方面， C_1 与 C_2 的内部密度明显高于其余社群；另一方面， C_4 与多个高密度社群都保持较高的跨社群连接密度，因此在整个校园网络中起到了明显的连接作用。

5.5 社群关系分析

在全部社群对中，跨社群连接最强的若干组合主要集中在 C_4 周围，说明 C_4 具有明显的桥梁作用。进一步只考察前 5 个高密度社群之间的联系，可得到表 7。

表 7: 前 5 个高密度社群之间的主要联系

社群对	跨社群边数	跨社群连接密度	解释
C_4-C_5	49	0.0790	两社群都带有明显的理工实践属性，成员在科技类活动和兴趣社团层面存在较强联系
C_2-C_4	27	0.0726	高年级理工群体与工科创新群体之间联系紧密，反映出实践项目与学长学姐资源的连接
C_1-C_4	21	0.0677	研究生计算机社群与工科创新社群之间存在技术协作关系，体现出知识与项目的流动
C_3-C_5	18	0.0600	文科兴趣群体与理工实践群体之间仍保持一定交流，说明平台并非完全按学科隔离
C_3-C_4	27	0.0581	文科新生社群与工科创新群体也存在较多连接，反映兴趣社团对跨专业社交的促进作用

从表 7 可以看出， C_4 与 C_5 、 C_2 、 C_1 均保持较强联系，因此 C_4 可视为连接不同高密度社群的重要社群。换言之，校园社交网络中的高密度社群并不是彼此完全分离，而是通过创新创业、电子科技、学生组织等活动形成了多条联系。

5.6 功能重叠分析

由于本文采用标准 Louvain 算法进行非重叠主社群划分，每个用户在模型求解中仅被赋予一个主社群标签。因此，在本文的主社群划分口径下，各社群成员集合互不

相交。为进一步回应题目中“功能定位是否存在重叠”的要求，本文对前 5 个高密度社群的年级分布、专业分布和社团分布分别构造向量，并计算功能重叠指数，结果如图 5 和表 8 所示。

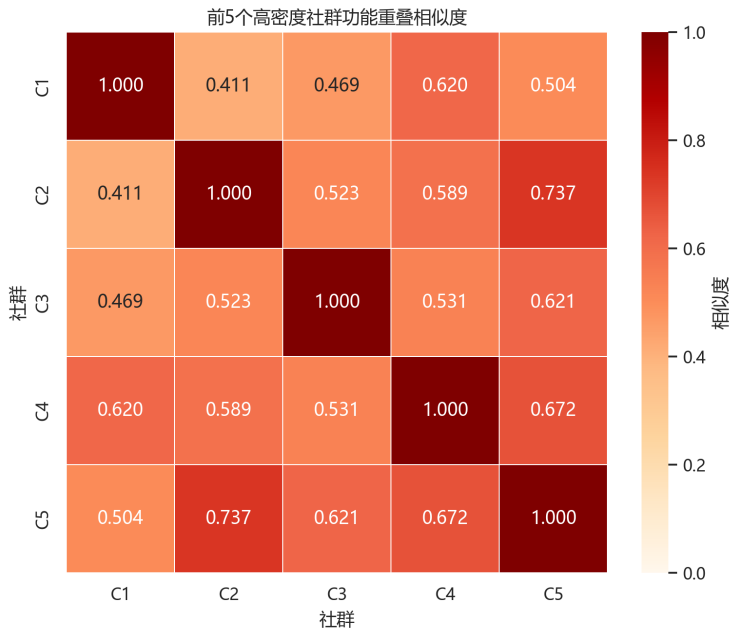


图 5：前 5 个高密度社群的功能重叠相似度热力图

图 5 用颜色深浅展示了前 5 个高密度社群两两之间的功能接近程度：主对角线元素恒为 1，表示社群与自身完全重合；非对角元素颜色越深、数值越大，说明两个社群虽然成员不重叠，但在年级结构、专业背景和社团参与上越相似。其中， C_2-C_5 、 C_4-C_5 和 C_1-C_4 的颜色相对更深，表明这些社群之间存在较强的功能邻近关系。这里所说的功能重叠与前文的 Louvain 主社群划分不是同一个层面的对象：Louvain 给出的主社群是非重叠的，每个用户只能属于一个主社群；功能重叠考察的则是不同社群在画像和作用上的相似程度，因此即使成员不交叉，也可能出现较高重叠。

表 8: 前 5 个高密度社群的功能重叠结果

社群对	年级相似度	专业相似度	社团相似度	功能重叠指数	解释
C_2-C_5	0.8301	0.6701	0.7099	0.7367	两社群在年级结构、理工背景和电子科技协会参与上都较接近，是最典型的科技实践相邻社群
C_4-C_5	0.9540	0.3786	0.6828	0.6718	年级结构和社团参与高度相似，但专业分布差异更大，表现出较强协作关系和专业互补性
C_3-C_5	0.8735	0.2873	0.7026	0.6211	虽然专业背景差异明显，但兴趣社团和活跃结构相近，说明兴趣圈层能够促成跨学科连接
C_1-C_4	0.7696	0.5526	0.5367	0.6196	两社群均带有较强技术属性，其中一个偏研究生技术核心，另一个偏本科创新协作平台
C_2-C_4	0.7147	0.3924	0.6588	0.5886	两社群都与科技实践活动密切相关，但在细分专业构成上仍存在一定差异

从图 5 和表 8 可以看出，前 5 个高密度社群确实存在明显的功能重叠，但这种接近主要体现在兴趣组织和年级结构上，不一定表现为细分专业的一致。特别地， C_2-C_5 的功能重叠指数最高，说明二者可以视为同一类科技实践圈层中的相邻社群； C_4-C_5 的年级相似度高达 0.9540，但专业相似度只有 0.3786，说明二者更接近跨专业协作关系，而不是同质群体； C_1-C_4 的技术背景也较接近，说明研究生技术社群与本科创新社群之间存在稳定联系。

需要说明的是，本文中的功能重叠特指不同主社群在年级、专业和社团三个画像维度上的相似性，并不意味着这些社群在现实中的社交功能完全一致。该指标的作用，是在非重叠主社群划分基础上，进一步刻画不同社群之间的功能接近与互补关系。

因此，本文认为前 5 个高密度社群在成员划分上无重叠，在功能定位上则存在可量化的交叉与互补：有的偏学术技术，有的偏工程实践，有的偏兴趣连接，它们共同构成了校园社交网络中的主要活跃圈层。

5.7 本问结论

综合来看，校园社交平台中的社群结构具有以下特点：

1. 网络可稳定划分为 10 个主社群，说明平台已经形成较清晰的群体结构；
2. 前 5 个高密度社群主要围绕研究生技术核心、理工实践、大一文科兴趣、工科创新枢纽和理工混合实践五类功能展开；

3. C_4 是起连接作用最明显的社群，在多个高密度社群之间承担连接作用；
4. 在 Louvain 非重叠主社群划分口径下，前 5 个高密度社群的成员归属互不交叉；但基于年级、专业和社团分布计算得到的功能重叠指数表明，不同主社群在功能定位上仍存在明显的交叉与互补。

六、第 2 问：面向 S11 的好友推荐模型

6.1 建模思路

第 2 问按照“数据输入 → 模型处理 → 输出结果”的思路展开：先从好友关系表、用户属性表和行为数据表中为 S11 构造非好友候选集；再提取结构、属性与行为三类特征，建立 SAB 综合评分模型；最后输出前 3 名推荐对象，并结合得分构成解释推荐理由与尚未成边的原因。

在这一框架下，第 2 问的目标是为 S11 推荐 3 位当前尚未成为好友的用户。由于推荐对象必须来自 S11 的非好友集合，因此首先需要构造候选集；又由于题目要求解释推荐原因和未成为好友的可能原因，模型就需要说明每个候选人在结构、属性和行为上的优势。相关文献在本文中主要起到两类作用：一类说明共同邻居、标签相似性、邻居重叠和局部结构信息是较稳定且易解释的链路预测依据^[6-9]，因此可作为推荐特征的主要来源；另一类以 Liben-Nowell 与 Adamic-Adar 为代表，给出了结构相似度与二阶关系刻画经典指标^[10,11]，为本文选取共同好友数、Jaccard 系数和 Adamic-Adar 指数提供了直接支撑。经典链路预测方法能够反映 S11 与候选用户是否处在相近社交圈中，但校园好友关系还会受到现实接触机会和平台使用习惯的共同影响。因此，本文在结构特征之外，还进一步结合 S11 的年级、专业与专业类别、社团参与、活跃时段、科技/文化话题偏好，以及互动频率、平台活跃度和发帖习惯等画像信息，构造面向特定用户的 SAB 推荐模型。

6.2 候选集构建

题目要求从非好友用户中筛选推荐对象，因此候选集应排除 S11 本人及其当前好友，只保留所有潜在成边对象。设用户 S11 的现有好友集合为 $N(S11)$ ，则推荐候选集定义为

$$\Omega = V \setminus (N(S11) \cup \{S11\}). \quad (6)$$

由数据可知，S11 的画像为：大一、历史学、文科方向，参加电子科技协会与机器人社团，下午时段活跃，文化类话题参与度显著高于科技类话题。因此，第 2 问不应只看网络结构，还应尽量推荐与其专业背景、兴趣组织和活跃习惯相匹配的候选用户。

6.3 特征设计与归一化

在第 6.2 节已经构造候选集的基础上，本文需要进一步判断每个候选用户与 S11 的匹配程度。结合题目给出的好友关系表、用户属性表和行为数据表，本文将匹配程

度分为结构层、属性层和行为层三类：结构层用于刻画两人在好友网络中的接近程度，属性层用于刻画现实身份背景与潜在接触机会，行为层用于刻画平台使用习惯与后续互动可能性。

对每个候选用户 $v \in \Omega$ ，本文构造以下三类特征：

- 结构特征：共同好友数、Jaccard 系数、Adamic-Adar 指数；
- 属性特征：共同社团数、是否同专业类别、是否同专业、是否同班、是否同年级；
- 行为特征：活跃时段是否一致、话题偏好相似度、互动频率相似度、平台活跃度相似度、发帖频率相似度。

其中，共同好友数、Jaccard 系数、Adamic-Adar 指数、共同社团数及若干差值型行为特征均采用极差归一化；对于二值变量，直接取 0/1；而科技类与文化类话题参与度能够直接反映两人在后续互动中的内容一致性，因此据此构造话题偏好相似度：

$$\tau_{S11,v} = 1 - \frac{|t_{S11}^{\text{tech}} - t_v^{\text{tech}}| + |t_{S11}^{\text{culture}} - t_v^{\text{culture}}|}{20}. \quad (7)$$

$\tau_{S11,v}$ 越大，说明两人在话题偏好上越接近。这里分母取 20，是因为科技类与文化类话题参与度均以 0 ~ 10 计量，单项绝对差的最大值为 10，两项绝对差之和的最大值为 20。因此除以 20 后，话题偏好相似度可稳定映射到 [0, 1] 区间，便于与其余归一化特征共同进入 SAB 评分模型。

6.4 SAB 综合评分模型

共同好友、Jaccard 系数和 Adamic-Adar 指数反映 S11 与候选用户在网络中的接近程度；共同社团、专业类别、班级和年级反映两人在现实校园中的接触机会；活跃时段、话题偏好、互动频率、平台活跃度与发帖习惯则反映后续互动持续发生的可能性。因此，第 2 问需要将网络接近性和现实匹配度统一放入同一评分框架。本文将这组结构、属性与行为特征的综合得分记为 Structure-Attribute-Behavior score，简称 **SAB 得分**。记归一化后的特征为 $x_i(S11, v)$ ，则

$$s_{S11,v}^{\text{SAB}} = \sum_{i=1}^{13} w_i x_i(S11, v). \quad (8)$$

式 (8) 给出了统一加权框架。本文依据题意和 S11 的画像信息对各项权重作经验设定：好友推荐首先要考虑现实接触机会，因此属性特征权重略高于结构特征；同时，S11 已经具有一定好友基础，二阶社交关系强弱仍是潜在成边的重要信号，因此结构权重与属性权重保持接近；相比之下，行为特征更容易受短期活跃波动影响，适合作为排序微调项，因此权重相对较低。基于这一判断，本文将大类权重按“结构 0.38: 属性 0.39: 行为 0.23”分配，大类内部再依据各子特征对 S11 画像的匹配敏感度和区分能力进一步细分配置。最终采用的主模型权重为：

- 结构特征共占 0.38：共同好友 0.18、Jaccard 0.12、Adamic-Adar 0.08；
- 属性特征共占 0.39：共同社团 0.15、同专业类别 0.08、同专业 0.07、同班 0.04、同年级 0.05；
- 行为特征共占 0.23：活跃时段一致 0.06、话题偏好相似 0.08、互动频率相似 0.04、平台活跃度相似 0.03、发帖频率相似 0.02。

这一设定使模型既能保留二阶好友关系的结构信息，又能体现 S11 在文科背景、社团兴趣与活跃习惯上的个性化需求。

6.5 求解流程与参数设置

第 2 问的程序实现同样基于 Python 展开，其中好友网络上的共同邻居、Jaccard 系数和 Adamic-Adar 指数计算主要依赖 NetworkX，原始 Excel 数据读取由 openpyxl 完成，特征归一化与结果导出则由共享数据处理模块配合 csv/json 模块完成。需要说明的是，第 2 问正文中的 SAB 主模型求解只涉及特征计算、归一化和线性加权排序，不包含随机搜索或随机仿真，因此推荐名单本身是确定性的；仅在第 9 章“已有好友-随机非好友”对照检验中额外使用了随机抽样，并固定随机种子为 42 以保证检验结果可复现。

第 2 问的求解流程如下：

1. 先按题意构造候选集 Ω ，即剔除 S11 本人及其当前好友后，对其余用户统一评分。本题中全网共 200 人，S11 已有 37 名好友，因此共有 162 名候选对象进入排序。
2. 对每个候选用户计算 13 个特征值，其中共同好友数、Jaccard、Adamic-Adar、共同社团数属于结构或属性强度指标；互动频率差、活跃度差和发帖频率差本质上刻画的是行为习惯的不匹配程度。对好友推荐而言，这类差异越小，越说明双方在交流节奏和平台使用方式上更接近，后续互动更容易持续，因此先取相反数，再做极差归一化，使“差异越小、得分越高”的含义得以保留。
3. 对连续型特征统一采用

$$x_i^*(v) = \frac{x_i(v) - \min_{u \in \Omega} x_i(u)}{\max_{u \in \Omega} x_i(u) - \min_{u \in \Omega} x_i(u)}$$

进行归一化；二值变量直接取 0/1；话题偏好相似度 $\tau_{S11,v}$ 已经被映射到 $[0, 1]$ 区间，可直接参与加权。

4. 将归一化后的特征代入 SAB 综合评分模型，按 0.38:0.39:0.23 的三类权重分配计算总分，并按得分从高到低进行排序；若得分相同，则按候选用户编号作稳定排序。

- 最后输出前 3 名候选用户，并同步提取其贡献最大的特征项，生成“为什么适合推荐”和“为什么当前尚未成为好友”的解释文本。

需要说明的是，仅含结构特征的模型和画像强化模型仅用于第 2 问的检验与对比，正文中给出的推荐名单来自上述 SAB 主模型的求解结果。

6.6 推荐结果

在 SAB 主模型下，S11 的前 10 名候选用户得分如图 6 所示。可以看出，得分前列并非都来自单一机制：有的依赖很强的结构关联，有的则来自更高的属性与行为匹配度。



图 6: S11 前 10 名候选在 SAB 模型下的综合得分

最终推荐的前 3 位候选用户如表 9 所示。

表 9: S11 的前 3 名推荐结果

排名	用户编号	综合得分	主要推荐依据	尚未成为好友的原因解释
1	S146	0.5717	同为历史学、同为大一、同在下 午活跃,且有 5 位共同好友,话 题偏好相似度达到 0.90	分属不同社群且不在同一班 级,同时缺少共同社团这一直 接触点
2	S24	0.5548	共享机器人社团与电子科技 协会,且有 7 位共同好友,同 属文科方向	分属不同社群,主要依赖共同 好友间接连接;对方更偏全天 活跃,存在时段错位
3	S2	0.5504	拥有 14 位共同好友, Jaccard 与 Adamic-Adar 指标均为全 体候选最高,下午活跃时段也 一致	专业背景和年级差异较大,且 没有共同社团,因此虽然二阶 关系强,但尚未形成直接好友 边

6.7 推荐结果分析

为进一步说明主模型为何得到上述排序，图 7 给出了前 3 名候选在结构、属性和行为三类特征上的贡献分解。

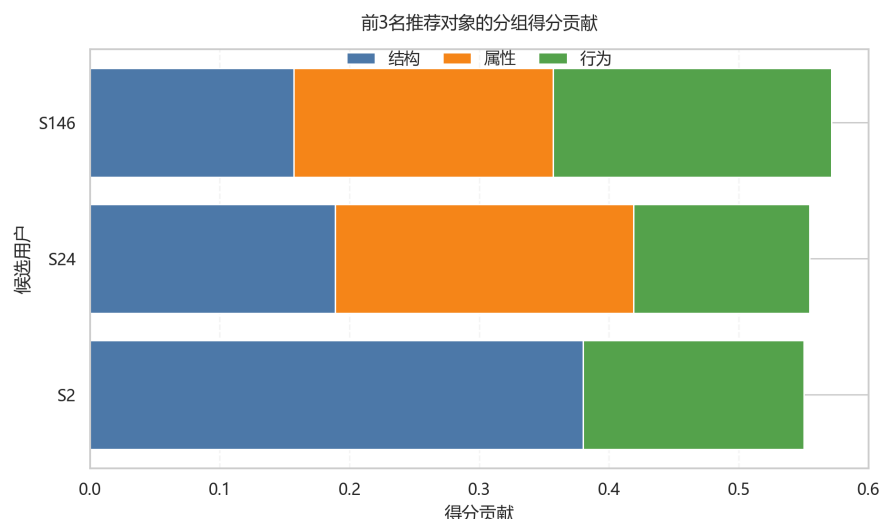


图 7: S146、S24、S2 分别体现三类推荐逻辑

从图 7 可以看出：

- S146 的属性与行为贡献最高，整体画像与 S11 最为接近。其与 S11 同专业、同年级、同活跃时段，且文化类话题偏好接近，因此虽然共同好友数不算最多，仍能取得最高综合得分。
- S24 的优势主要体现在共同社团和共同好友上。它同时连接了电子科技协会与机器人社团这两个 S11 最关键的兴趣组织，因此具备较强的线下接触可能性和线上交流基础。
- S2 更体现跨圈层连接作用。其与 S11 有 14 位共同好友，说明两人位于高度重叠的二阶社交圈中；尽管专业背景差异较大，但从平台撮合角度看，S2 仍是具有较高成边潜力的候选。

另外，排名第 4 的 S34 与主模型前三名非常接近。该用户与 S11 共享机器人社团，且话题偏好相似度达到 0.90，说明在更强调用户画像的权重设定下，S34 也会成为较强的备选推荐对象。

6.8 本问结论

综合来看，S11 的推荐结果来自三种不同的潜在成边路径：

1. S146 与 S11 在专业、年级和活跃习惯上都较为接近，更适合满足同圈层的精准推荐需求；
2. S24 依托共同社团和共同好友形成较强的潜在接触场景，线下见面与线上互动机会都较多；
3. S2 虽然画像差异较大，但二阶社交关系很强，说明跨圈层推荐同样可能形成有效连接。

因此, 本文认为面向 s_{11} 的好友推荐应综合结构、属性与行为三方面信息, 在同圈层匹配和跨圈层连接之间取得平衡。

七、第 3 问: 关键用户筛选与传播仿真

7.1 建模思路

第 3 问按照“数据输入 → 模型处理 → 输出结果”的思路展开: 先从好友网络和行为数据中筛出科技话题候选用户, 并计算静态传播潜力指标; 再结合题目给定的时序规则与转发概率进行 Monte Carlo 仿真, 形成联合得分; 最后输出关键用户排序, 并给出 48 小时传播过程与统计结果。

在这一框架下, 第 3 问要找的是在科技类话题下更具传播潜力、且能够在 48 小时内较快带动扩散的关键发帖用户, 而不是只比较谁的度数最大。传播动力学综述指出, 信息扩散结果既受网络拓扑影响, 也受传播时序机制影响^[12]; 社会网络节点影响力分析研究则表明, 关键用户识别不能只依赖单一度指标^[13]; 影响力传播研究还进一步说明, 核心层级与传播潜力之间存在紧密联系^[14,15]; 而经典影响力最大化模型进一步给出了从传播收益角度评价种子节点的基本框架^[16]。基于此, 本文将关键用户识别分为两步:

1. 先根据网络结构、行为强度和话题相关性对候选用户进行预评分;
2. 再结合题目给出的转发概率公式, 模拟其在正午 12:00 发帖后的真实传播过程, 并以传播结果反向校正排序。

7.2 候选用户筛选

考虑到题目明确指定的是“科技类话题”, 科技类话题参与度可以直接代表用户对该主题的基础投入意愿。若某用户对科技类内容本身几乎没有参与意愿, 则即使网络位置较好, 也不宜直接视为该主题下的关键用户。因此, 本文以样本中位数 5 作为最低话题门槛构造候选池。一方面, 这可以排除明显缺乏主题兴趣的用户; 另一方面, 也能避免候选池过大或过小, 从而让后续传播仿真保持较好的稳定性:

$$\Omega_3 = \{u \in V \mid A_u \geq 5\}, \quad (9)$$

其中 A_u 表示用户 u 在科技类话题上的参与度。由数据可得, 共有 122 名用户进入该候选池。

7.3 候选用户预评分

由于发帖时刻已经固定为正午 12:00, 发帖者自身的活跃时段不再作为直接优势; 相反, 更重要的是其好友在该时刻之后能否尽快看到帖子。直接触达范围、跨社群桥接能力、网络核心程度、互动黏性、科技话题相关度和受众时序匹配, 共同决定某个

用户是否适合作为传播种子。因此，本文先将各项指标统一归一化到 $[0, 1]$ ，再采用等权加权构造候选用户预评分：

$$S_u = \frac{1}{8} (D_u + B_u + C_u + K_u + \tilde{F}_u + N_u + \tilde{A}_u + Q_u). \quad (10)$$

上式中的各项指标均已事先归一化到 $[0, 1]$ 区间，因而 $\tilde{A}_u$ 与 $\tilde{F}_u$ 在预评分阶段表示的是归一化后的科技参与度与互动频率指标。式 (10) 采用等权平均，原因有二：其一，这 8 项指标量纲已统一，直接取平均不会受到尺度差异干扰；其二，在进入传播仿真前，很难仅凭静态特征就断言某一项应长期压过其他指标。因此，本文将等权预评分作为候选筛选阶段的中性起点，再由后续传播结果进一步拉开差距。各项指标分别表示：

- D_u ：度中心性，反映直接触达好友的规模；
- B_u ：介数中心性，反映跨圈层桥接能力；
- C_u ：接近中心性，反映其在网络中的平均距离优势；
- K_u ： k -core 指标，反映其所处核心层次；
- $\tilde{F}_u$ ：归一化后的平均互动频率，反映其互动黏性；
- N_u ：发帖频率，反映其平台发声活跃度；
- $\tilde{A}_u$ ：归一化后的科技类话题参与度；
- Q_u ：受众时序匹配得分。

进一步地，等权预评分用于在缺少外部先验权重的情况下，先根据网络结构、行为强度和主题相关性筛出较有潜力的候选用户；最终排序再由传播仿真结果校正，以避免单一静态指标直接决定关键用户。

其中， Q_u 用于刻画候选用户在正午发帖后，其好友整体能多快完成首轮看到。若某位好友越早看到帖子，就越可能推动前 12 小时内的扩散启动，因此本文先把每位好友的可见延迟转换为一个 $[0, 1]$ 得分，再对全部好友取平均：

$$Q_u = \frac{1}{|N(u)|} \sum_{v \in N(u)} \left(1 - \frac{\delta_v}{22}\right), \quad (11)$$

其中 $\delta_v \in \{0, 3, 8, 22\}$ 分别对应“全天、下午、晚上、上午”四类活跃时段在正午发帖后的最短可见延迟。之所以用 $1 - \frac{\delta_v}{22}$ ，是因为四种时段中最慢的情况恰好是“上午活跃”用户需要等到次日 10:00 才能看到帖子，对应最大延迟 22 小时；据此可把“越早看到越有利”稳定映射到 $[0, 1]$ 区间，其中全天活跃得分最高，上午活跃得分最低。再除以 $|N(u)|$ ，是为了对全部好友取平均，避免度数较大的用户仅凭好友数多而重复占优。于是， Q_u 实际衡量的是“从好友视角看，用户 u 在正午发帖后能多快被第一轮看见”， Q_u 越大，说明其好友越容易在发帖后的当日下午或当晚完成首轮触达。

7.4 传播机制与 Monte Carlo 仿真

本文将好友关系视为无向传播网络，并按题意设置如下规则：

- 非好友无法直接看到该帖；
- 好友会在自己的活跃时段首次看到该帖或转发帖；
- 若用户多次看到同一话题，则每次曝光的转发尝试相互独立，但每名用户最多转发一次。

设某位好友在时刻 t 转发帖子，则用户 v 的看到时刻记为 $\tau(v, t)$ ，定义为

$$\tau(v, t) = \begin{cases} t, & v \text{ 全天活跃,} \\ \min\{s \geq t \mid 12 + s \equiv 15 \pmod{24}\}, & v \text{ 下午活跃,} \\ \min\{s \geq t \mid 12 + s \equiv 20 \pmod{24}\}, & v \text{ 晚上活跃,} \\ \min\{s \geq t \mid 12 + s \equiv 10 \pmod{24}\}, & v \text{ 上午活跃.} \end{cases} \quad (12)$$

这一定义特别体现了题目中的关键细节：若原帖在正午 12:00 发出，而某位用户只在上午活跃，则其首次看到时间必然为次日 10:00，即 22 小时后。

在转发阶段，科技参与度反映用户对该主题的主观兴趣，互动频率反映其日常互动强度，而曝光延迟则反映热度随时间衰减的现实过程，因此题目给出的转发概率公式可以直接用来描述单次曝光后的转发倾向：

$$P_{\text{share}}(u, \Delta t_u) = \frac{1}{1 + \exp \left[- \left(-3.5 + 3.0 \cdot \frac{A_u}{10} + 2.0 \cdot \frac{F_u}{25} - 0.08 \Delta t_u \right) \right]}, \quad (13)$$

其中 A_u 为科技类话题参与度， F_u 为平均互动频率， Δt_u 为从原帖发布时间到用户 u 看到当前曝光的时间差。这里的 A_u 与 F_u 沿用题目给定公式中的原始记号，代入的是行为数据表中的原始数值，而不是前述预评分中的归一化指标。由于科技类话题参与度的取值范围为 $0 \sim 10$ ，故用 $A_u/10$ 将其缩放到 $[0, 1]$ 附近；平均互动频率在样本中的上界为 25，故用 $F_u/25$ 做同量纲缩放。相比之下，第 2 问中的相似度分母 20 来自两个话题差值上界的相加，因此三者虽然写法不同，但都服务于同一个目的：在进入评分或概率函数前先完成合理的尺度统一。可见，科技参与度和互动频率会提升转发倾向，而曝光延迟越大，转发概率越低。

为同时考虑传播速度与最终覆盖规模，本文不只看 48 小时终值，而是把 6h、12h、24h、48h 的覆盖情况和 48h 的转发情况合并成传播表现分值。其中，12h 覆盖权重设为最高的 0.25，因为不同种子用户的差异主要体现在前半天到一天内的启动速度；6h 覆盖只赋权 0.15，是因为该时段过短、随机波动更大；24h 覆盖、48h 覆盖和 48h 转发

各赋权 0.20，用于共同刻画中后期的覆盖规模与继续扩散能力。于是定义候选用户 u 的传播表现分值为

$$M_u = 0.15 \frac{\bar{R}_u^{\text{cov}}(6)}{200} + 0.25 \frac{\bar{R}_u^{\text{cov}}(12)}{200} + 0.20 \frac{\bar{R}_u^{\text{cov}}(24)}{200} + 0.20 \frac{\bar{R}_u^{\text{cov}}(48)}{200} + 0.20 \frac{\bar{R}_u^{\text{share}}(48)}{200}, \quad (14)$$

其中 $\bar{R}_u^{\text{cov}}(t)$ 表示在 t 小时内累计看到该帖的平均用户数， $\bar{R}_u^{\text{share}}(t)$ 表示累计完成转发的平均用户数。每一项都除以 200，是因为全网总人数为 200，这样不同时间点的覆盖与转发规模可以统一到相近尺度上进行比较。最终关键用户得分定义为

$$Z_u = 0.35S_u + 0.65M_u. \quad (15)$$

本文对候选池中的每位用户进行 240 次 Monte Carlo 仿真，得到初始排序；之所以选择这一重复次数，是为了在计算成本可控的前提下先筛掉明显不适合的用户。随后，考虑到前 3 名用户之间的得分差距较小，本文再对最终前 3 名用户追加 420 次仿真，以降低随机波动并获得更平滑的扩散曲线和统计量。

7.5 联合求解流程与参数设置

第 3 问的程序实现仍基于 Python 展开，其中候选用户的度中心性、介数中心性、接近中心性和 k -core 等静态指标主要依赖 NetworkX 计算，传播过程采用 `heapq` 优先队列实现事件驱动模拟，Monte Carlo 重复仿真与统计汇总则分别调用 `random` 和 `statistics` 模块完成。程序先对候选池内全部用户执行 240 次基准场景仿真，再对最终前 3 名执行 420 次追加仿真；为保证不同候选用户之间的传播结果可复现，仿真过程中对每位候选用户都采用按用户编号构造的固定基础随机种子。

第 3 问的完整求解过程可概括为“候选筛选–静态预评分–动态仿真–联合排序”四步：

1. 先以科技话题参与度阈值 $A_u \geq 5$ 筛出候选池，共保留 122 名用户。该阈值对应样本中位数，可避免明显缺乏科技话题兴趣的用户进入关键用户候选集。
2. 对候选池中每名用户计算 8 个归一化指标，即度中心性、介数中心性、接近中心性、 k -core、平均互动频率、发帖频率、科技话题参与度和受众时序匹配得分，并以等权方案 1/8 计算静态种子得分 S_u 。熵权法仅在模型检验部分作为对照，不参与主排序。
3. 对每名候选用户都以正午 12:00 发帖为起点，在基准传播参数下进行 240 次 Monte Carlo 仿真，统计 6h、12h、24h、48h 的平均覆盖人数与平均转发人数，再据此计算传播表现分值 M_u 。
4. 按 $Z_u = 0.35S_u + 0.65M_u$ 得到最终得分，并按“最终得分优先、48h 平均覆盖次之”的规则排序。考虑到前 3 名之间差距较小，本文再对最终前 3 名追加 420 次仿真，用于生成正文中的传播曲线和 95% Monte Carlo 区间。

因此，第 3 问的结果来自候选筛选、指标归一化、多轮仿真和联合排序后的综合判定。

7.6 关键用户排序结果

按最终得分 Z_u 排序后的前 10 名关键用户如图 8 所示。可以看出，进入前列的用户并非只依赖单一指标，而是在网络位置、科技话题相关性和好友首轮可见速度三方面取得了更好的平衡。

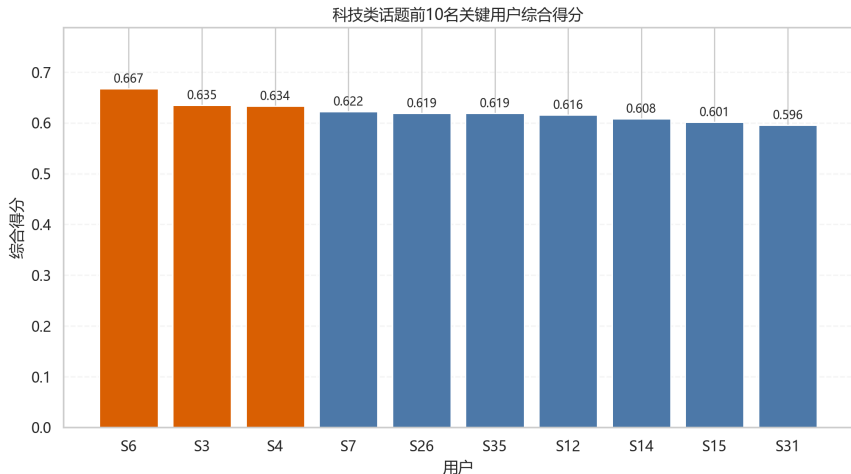


图 8: 科技类话题前 10 名关键用户最终得分

最终排名前 3 的关键用户如表 10 所示。

表 10: 科技类话题前 3 名关键用户

排名	用户编号	最终得分 Z_u	科技参与度 A_u	12h 覆盖	48h 覆盖	48h 转发
1	S6	0.6673	8	167.05	199.13	106.09
2	S3	0.6353	9	158.68	198.24	101.12
3	S4	0.6336	7	163.25	199.10	103.36

表 10 反映了三个层次不同的关键用户：

- S6 的综合表现最好。其度数达到 59，科技参与度为 8，且好友侧首轮触达优势最强，因此在本文设定下得分最高。
- S3 的科技参与度达到 9，互动频率也较高，属于兼顾主题契合度与扩散能力的平衡型关键用户。
- S4 的传播覆盖能力很强，纯传播指标与 S6 十分接近，但科技话题参与度略低于 S3，因此综合排序稍后。

另外，S4 在 12 小时平均覆盖上高于 S3。但第 3 问关注的是“科技类话题”的关键用户，而不仅是一般意义上的传播中心，因此 S3 依靠更高的科技参与度和互动频率获得了更高的最终排序。这也说明本文模型同时考虑了覆盖规模与话题契合度。

7.7 48 小时传播结果

图 9 给出了前 3 名关键用户的平均覆盖曲线。整体上看，三条曲线在前 12 小时差异最明显，之后逐渐趋同；这意味着第 3 问中区分关键用户优劣的，不只是 48 小时终值，还包括首轮和第二轮扩散的启动速度。

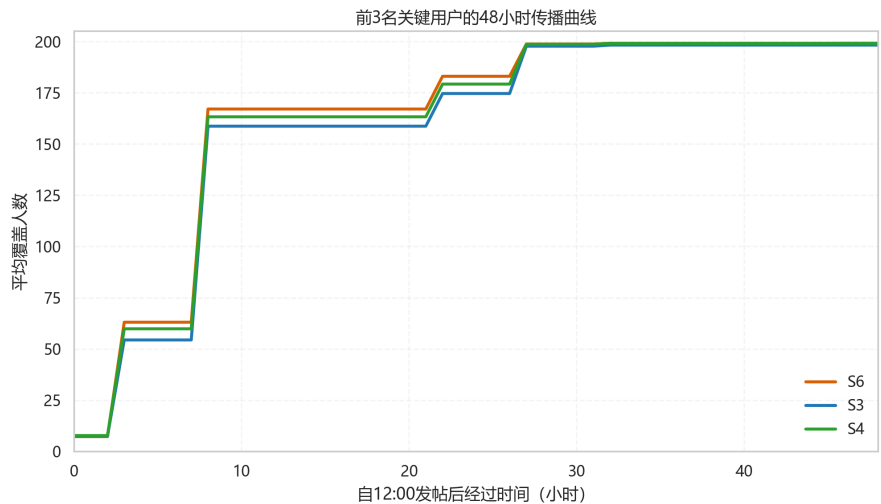


图 9: 前 3 名关键用户的 48 小时平均覆盖曲线

综合得分最高的 S6 在 420 次追加仿真下的传播过程如图 10 所示。可以看出，覆盖人数曲线在前 12 小时上升最快，而转发人数曲线在 24 小时后趋于平缓，符合“早期扩散决定最终规模”的传播特征。

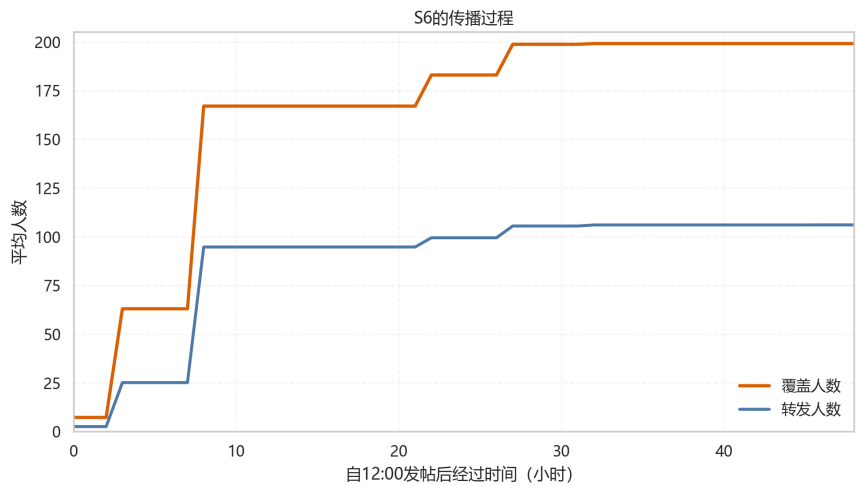


图 10: S6 在 48 小时内的平均覆盖与转发曲线

对 S6 的关键时间点统计如表 11 所示。

表 11: S6 的传播仿真统计结果

指标	6h	12h	24h	48h
平均覆盖人数	63.04	167.05	183.02	199.13
覆盖 95% 区间	[32.00, 76.53]	[142.00, 179.00]	[158.00, 195.00]	[196.00, 200.00]
平均转发人数	25.14	94.74	99.49	106.09
转发 95% 区间	[4.00, 42.00]	[68.00, 119.00]	[73.00, 125.53]	[81.48, 129.00]

由表 11 可知, S6 发帖后 12 小时内平均即可覆盖 167.05 名用户, 已达到全网规模的 83.5%, 其覆盖人数的 95% Monte Carlo 区间为 [142.00, 179.00]; 到 48 小时时, 平均覆盖人数达到 199.13, 对应区间为 [196.00, 200.00], 几乎完成全网触达。同时, 平均转发人数达到 106.09, 其 48 小时转发人数区间为 [81.48, 129.00], 说明其不仅覆盖人数多, 而且继续转发的人也多。

若仅采用单指标基线, 结果会明显逊色。例如, 科技参与度最高的基线用户 S14 在 12 小时和 48 小时的平均覆盖分别为 146.57 和 196.85; 平均互动频率最高的基线用户 S126 对应指标仅为 106.87 和 155.99。因此, 单纯依赖话题相关度或互动频率都不足以识别适合启动扩散的关键用户。

7.8 本问结论

综合预评分与传播仿真结果, 本文认为科技类话题下表现较好的关键发帖用户为 S6, 备选关键用户为 S3 与 S4。其中:

- S6 在网络位置、话题相关度和首轮触达速度上都表现突出, 因此综合表现最好;
- S3 的话题契合度更高, 各项指标较为均衡;
- S4 的传播速度较快, 但主题相关度略弱。

因此, 第 3 问的关键用户识别应将网络桥接能力、科技话题相关性和活跃时序机制放在同一框架下综合评估。

八、 第 4 问: 定向推送配额约束下的传播优化

8.1 建模思路

第 4 问按照“数据输入 → 模型处理 → 输出结果”的思路展开: 先依据活跃度与文化话题参与度筛出可投放候选池, 并结合社群与行为指标刻画候选用户; 再在两阶段配额约束下, 用阶段贪心和局部交换搜索近似最优推送方案; 最后输出最优投放名单及其与多类基线策略的传播效果比较。

选做题要求平台围绕一条“文化类话题”信息, 在有限推送名额约束下设计分批、分时段的推送策略, 使其在发帖后 48 小时内的传播范围最大。题目已说明被系统推送的用户会在推送时刻立刻看到, 因此第 4 问可以表述为“带阶段配额约束的影响力最

大化问题”。其中，整体目标函数的设定与经典影响力最大化模型一致^[16,17]，而候选用户的预筛选与补入跨社群用户的思路则与节点影响力分析研究的结论相吻合^[13]。

8.2 候选池构建与阶段划分

考虑到题目明确将推送对象限定为“活跃用户”，且文化类话题的首轮触达效果依赖用户本身的活跃水平与主题兴趣，因此本文用活跃度和文化类话题参与度共同刻画候选资格，将满足

$$\text{activity_count}_u \geq 5, \quad \text{culture_topic}_u \geq 5 \quad (16)$$

的用户定义为可投放候选用户。这里的阈值 5 取自样本中位数，一方面可排除明显缺乏活跃度或文化兴趣的用户，另一方面也能避免候选池过小而导致策略比较不稳定。按此规则从全网 200 名学生中筛得 68 名候选对象，既保证其具备较高活跃度，也保证其对文化类内容具有基本兴趣。

又因为平台每天提供 10 个推送名额，研究窗口为 48 小时，故本文将推送过程划分为两个阶段：

$$t_1 = 0\text{h}, \quad t_2 = 24\text{h}, \quad (17)$$

每个阶段最多投放 10 人，总推送名额为 20。之所以这样划分，是因为 48 小时窗口天然对应两轮日内运营节奏：第一轮用于启动扩散，第二轮则依据前一日的扩散态势进行补点与修正。

8.3 优化模型

在有限推送配额下，第 4 问的核心目标就是尽可能扩大 48 小时内的触达人数，因此设阶段化推送策略记为

$$\mathcal{S} = \{(t_k, u_k) \mid k = 1, 2, \dots, 20\},$$

其中 u_k 表示被推送用户， $t_k \in \{0, 24\}$ 表示推送时刻。优化目标为最大化 48 小时内的期望触达规模

$$\max_{\mathcal{S}} F(\mathcal{S}) = \mathbb{E}[N_{48}(\mathcal{S})], \quad (18)$$

并满足

$$\sum_{u \in V} \mathbf{1}\{(0, u) \in \mathcal{S}\} \leq 10, \quad \sum_{u \in V} \mathbf{1}\{(24, u) \in \mathcal{S}\} \leq 10. \quad (19)$$

为了在 68 名候选对象中快速找到兼顾传播规模和解释性的方案，需要先用能够代表直接触达、桥接能力、网络核心位置、互动黏性、活跃程度、主题相关性和跨社群覆盖的指标构造预评分：

$$\phi(u) = \frac{\tilde{d}_u + \widetilde{BC}_u + \widetilde{CC}_u + \tilde{K}_u + \tilde{I}_u + \widetilde{Ac}_u + \tilde{H}_u + \tilde{B}_u}{8}, \quad (20)$$

各项指标说明如下：

- $\tilde{d}_u$ ：归一化后的度数，反映其直接触达好友的规模；
- $\widetilde{BC}_u$ ：归一化后的介数中心性，反映跨社群桥接能力；
- $\widetilde{CC}_u$ ：归一化后的接近中心性，反映其在网络中的平均距离优势；
- $\tilde{K}_u$ ：归一化后的 k -core 指标，反映其所处核心层次；
- $\tilde{I}_u$ ：归一化后的平均互动频率，反映其互动黏性；
- $\widetilde{Ac}_u$ ：归一化后的活跃度，反映其平台活动参与水平；
- $\tilde{H}_u$ ：归一化后的文化类话题参与度，反映其对文化类内容的兴趣程度；
- $\tilde{B}_u$ ：归一化后的跨社群桥梁度，反映其好友在不同社群中的分布比例。

预评分不直接决定最终策略，而是用于刻画候选用户的综合传播潜力。

由于两天共 20 个推送名额需要从活跃候选池中组合选择，若直接穷举全部可能方案，计算规模过大；若只采用纯贪心选择，又容易在早期决定后陷入局部最优。因此，在求解层面，本文采用**阶段贪心与局部交换**的两步法：

1. 先在每个阶段内逐步加入边际增益最大的候选用户，即对当前策略 $\mathcal{S}$ 选择

$$\Delta(u \mid \mathcal{S}) = F(\mathcal{S} \cup \{(t, u)\}) - F(\mathcal{S}) \quad (21)$$

最大的用户；

2. 再从最优初始策略出发，执行单点交换局部搜索，用新的候选用户替换已选用户，只要 48 小时传播规模不下降且早期传播表现更优，就接受该替换。

其中 $F(\mathcal{S})$ 由离散传播仿真给出，传播概率沿用第 3 问的题面公式，并在每个策略下进行 420 次 Monte Carlo 重复模拟，以在计算成本可控的前提下尽量减小随机波动。

8.4 求解流程与参数设置

第 4 问的程序实现继续基于 Python 展开，其中候选用户的中心性指标、 k -core 分解和跨社群桥梁度计算主要依赖 NetworkX，分阶段贪心与局部交换中的传播评估采用 `heapq` 维护事件队列，并通过 `random` 和 `statistics` 模块完成 Monte Carlo 仿真与统

计汇总。为保证不同阶段和不同基线策略之间的传播效果可以直接比较，程序在分阶段选取、局部交换和最终策略比较中均固定基础随机种子；其中快速边际收益评估采用 36 次仿真，局部交换评估采用 60 次仿真，正文中的最终策略比较统一采用 420 次仿真。

为了得到可执行的投放方案，本文在程序中采用如下求解流程：

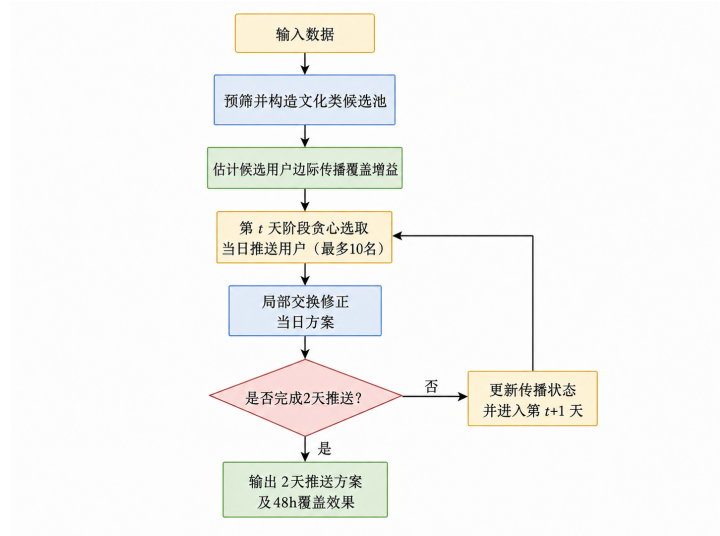


图 11: 第 4 问两阶段定向推送优化的核心流程图

图 11 直接对应了第 4 问的核心算法流程：先输入好友关系、用户属性和行为数据，预筛并构造文化类候选池；再估计各候选用户的边际传播覆盖增益，在第 t 天阶段内用贪心法选出当日最多 10 名推送对象，并通过局部交换对当日方案作进一步修正；若尚未完成两天推送，则更新传播状态后进入下一天，最终输出两天推送方案及其 48 小时覆盖效果。该流程与前文“候选池构建与阶段划分”“优化模型”和后文程序实现保持一致，体现了本文从候选筛选、阶段选取到状态更新的完整求解过程。

1. 先按“活跃度不低于 5 且文化话题参与度不低于 5”筛出 68 名候选用户，并根据度数、介数中心性、接近中心性、 k -core、互动频率、活跃度、文化话题参与度和桥梁度计算预评分 $\phi(u)$ ，作为后续搜索的候选优先级。
2. 在第 1 阶段 $t_1 = 0h$ 和第 2 阶段 $t_2 = 24h$ 内，分别执行阶段贪心选择：每次都在尚未入选的候选用户中，寻找能够使 48 小时平均覆盖增量 $\Delta(u | \mathcal{S})$ 最大的对象加入当前方案。为了控制计算成本，每次边际收益比较采用 36 次 Monte Carlo 快速评估。
3. 得到若干可解释初始方案后，再选取传播目标值最好的方案作为局部搜索起点，执行单点交换改进。若用一个新用户替换当前方案中的某个用户后，能使“48h 平均覆盖优先、24h 平均覆盖次之、48h 平均转发再次之”的目标元组变大，则接受该替换。局部交换阶段统一采用 60 次仿真评估。

4. 最终将优化后的方案与纯贪心、最高度数、最高活跃度、最高文化参与度等基线在同一随机种子下各评估 420 次，得到正文中的 12h、24h、48h 传播结果与区间。据此，第 4 问在配额约束下通过阶段贪心和局部搜索完成近似求解。

8.5 策略比较结果

在相同候选池、相同仿真次数和相同随机种子条件下，本文将优化策略与纯贪心策略、最高度数策略、最高活跃度策略以及最高文化参与度策略进行比较，结果如表 12 所示。

表 12: 第 4 问不同推送策略的传播效果比较

策略	12h 平均覆盖	24h 平均覆盖	48h 平均覆盖	48h 区间
本文优化策略	180.07	196.78	199.80	[199, 200]
纯贪心策略	179.59	195.63	199.67	[198, 200]
最高度数策略	177.95	193.81	199.35	[198, 200]
最高活跃度策略	176.82	192.73	198.26	[197, 200]
最高文化参与度策略	179.99	196.69	199.78	[198, 200]

由表 12 可知，本文优化策略在 48 小时目标下取得了最高的平均传播规模 199.80 人，相比最高度数策略提升 0.4548 人，相比最高活跃度策略提升 1.5405 人；即便与最强基线“最高文化参与度策略”相比，仍有 0.0167 人的平均优势。由于全网总人数仅为 200，这表明优化方案已十分接近该数据集上的传播上界。

图 12 更直观地展示了不同策略在 48 小时目标下的比较结果。可以看到，单纯依赖活跃度排序会明显损失跨社群覆盖能力，而仅依赖度数排序也无法充分利用主题相关性；同时考虑主题相关性、跨社群作用和阶段投放后，整体传播效果最好。

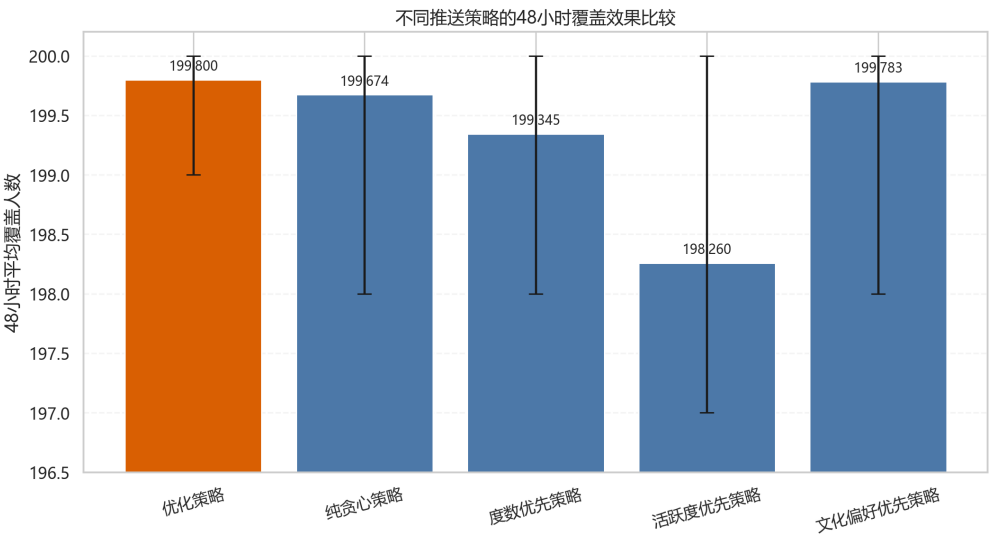


图 12: 本文优化策略在 48 小时传播目标下优于各类可解释基线

8.6 优化策略结构分析

最终优化方案的 20 个推送对象分布在 8 个社群中，其中第 9 社群选中 4 人，第 3、7、8、10 社群各选中 3 人，第 5 社群选中 2 人，其余社群少量补点。其活跃时段组成表现为：下午活跃用户 13 人、晚上活跃用户 6 人、全天活跃用户 1 人；入选用户的平均活跃度为 8.45，平均文化话题参与度为 9.25。这说明在文化类话题的即时推送场景下，较高的主题相关性和下午时段匹配对首轮扩散尤为重要。

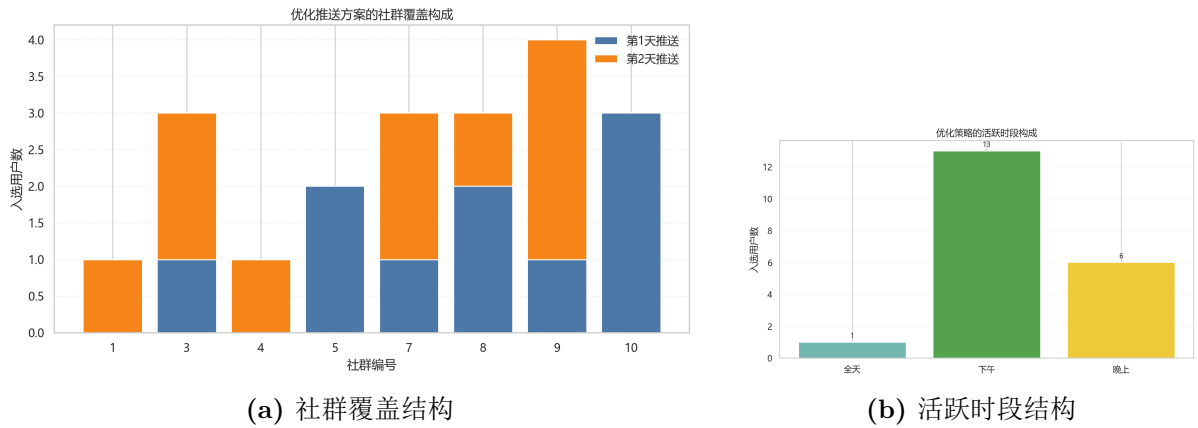


图 13: 优化推送策略的结构画像

进一步结合局部搜索过程可以发现，最优初始方案实际来自最高文化参与度策略，随后再用单点交换补入跨社群作用更强、覆盖更广的用户，最终得到更优的组合。这说明对于选做题而言，主题相关性是入选的重要条件，而跨社群扩散能力决定了是否还能在接近饱和时继续获得边际收益。

8.7 本问结论

第 4 问的最终结论是：在本题数据下，将推送对象限定为活跃且对文化话题敏感的候选池后，先构造高相关初始解，再结合阶段贪心与局部交换进行优化，能够在本文比较的方案中取得最好的传播效果，并在 48 小时内实现平均 199.80 人的传播覆盖，已非常接近全网覆盖上限。结果表明，推送对象既要保持较高的主题相关性，也要兼顾跨社群连接作用与时段匹配。这一结果可为校园平台在有限推送预算下进行文化类内容运营提供参考。

九、模型检验与敏感性分析

为便于整体把握四问结论的可信度，本文先将各问采用的主要检验方式、关键结果及最终判断汇总如表 13 所示，随后再分别展开说明。

表 13: 四问模型检验结果总览

问题	检验方式	关键结果	结论
第 1 问	Louvain 与 greedy modularity 社群划分对比, 并对功能重叠权重做轻度扰动	模块度分别为 0.247199 与 0.246760; 在 19 组权重扰动下, C_2-C_5 始终排第 1, C_4-C_5 始终排第 2	样本网络存在清晰社群结构, 且前 5 社群的主要功能重叠关系不依赖等权设定
第 2 问	结构模型、SAB 主模型与画像强化模型对比, 并做“已有好友-随机非好友”对照	主模型前 3 与结构模型仅重合 1/3; 好友组平均 SAB 得分 0.3758, 高于随机非好友对照的 0.2569, AUC 为 0.7926	属性与行为信息会显著影响个性化推荐排序, SAB 得分也能识别真实已成边关系
第 3 问	等权方案与熵权方案对比, 并对概率参数做 $\pm 10\%$ 扰动	两种权重方案下第 1 名均为 S6; 在保守、基准、乐观场景下, 其 48h 平均覆盖分别为 196.82、199.13、199.82	关键用户排序和传播结论较稳定, S6 的第 1 名不受轻度扰动影响
第 4 问	优化策略与纯贪心、最高度数、最高活跃度、最高文化参与度基线对比	本文策略在 24h 与 48h 指标上均最优, 其中 48h 平均覆盖达到 199.80	优化方案在接近饱和的传播区间内仍有小幅但稳定的优势

9.1 第 1 问模型检验

本检验旨在验证样本网络是否在不同社群发现算法下仍表现出明显社群结构, 并说明 Louvain 更适合识别题目关注的高密度社群; 同时检验前 5 个高密度社群的功能重叠结论是否由等权设定偶然造成。为此, 本文先将主模型 Louvain 与 greedy modularity 划分结果进行对比, 结果如表 14 所示。

表 14: 第 1 问社群划分稳定性检验结果

指标	数值
Louvain 社群数	10
Louvain 模块度	0.247199
Greedy modularity 社群数	6
Greedy modularity 模块度	0.246760
两种划分的节点对一致率	0.770402
Louvain 前 5 社群平均密度	0.290358
Greedy 前 5 社群平均密度	0.183346

由表 14 可以看出, 两种社群算法得到的模块度非常接近, 说明它们对网络整体社群结构的判断是一致的; 同时, 节点对一致率达到 0.770402, 表明绝大多数节点对在两种划分下具有相近的聚类关系。进一步地, Louvain 划分得到的前 5 个高密度社群平均密度更高, 说明其更有利于识别题目所关注的“内部连接紧密”的核心群体。因此, 本文采用 Louvain 结果作为第 1 问的最终划分方案是合理的。

在固定 Louvain 主社群划分不变的前提下，本文进一步检验“功能重叠”结论是否依赖于年级、专业和社团三类画像维度的等权设定。具体做法是：以等权值 $1/3$ 为中心，将每一维权重限制在 $[0.2333, 0.4333]$ 区间内，并按 0.05 步长枚举满足三者和为 1 的可行组合，共得到 19 组轻度扰动方案，结果如表 15 所示。

表 15: 第 1 问功能重叠权重扰动检验结果

指标	结果
扰动方案数	19 组
权重扰动范围	年级、专业、社团三类权重均在 $[0.2333, 0.4333]$ 内变动，步长为 0.05
功能重叠指数第 1 名	C_2-C_5 ，在 $19/19$ 组方案中保持不变
功能重叠指数第 2 名	C_4-C_5 ，在 $19/19$ 组方案中保持不变
第 3 名变化情况	仅在 C_3-C_5 与 C_1-C_4 之间切换，其中前者出现 10 次，后者出现 9 次

由表 15 可见，在轻度权重扰动下，前两组主要功能重叠关系始终稳定，即 C_2-C_5 始终是最强的功能相邻社群， C_4-C_5 始终保持第二。只有第 3 名在 C_3-C_5 与 C_1-C_4 之间发生局部互换，说明边界位置相近的候选社群对会受到权重微调影响，但“前 5 个高密度社群之间确实存在显著功能重叠，且最主要的重叠关系不依赖等权假设”这一结论较稳定。

9.2 第 2 问模型检验

本检验旨在验证第 2 问的推荐排序是否由单一模型选择偶然造成，并检验 SAB 得分是否能够区分真实已成边关系与随机非好友对照。若仅保留结构特征，模型容易偏向共同好友较多的候选用户；若过度强调画像匹配，又可能忽略网络中的真实社交接近程度。因此，本文先通过不同权重方案对比完整 SAB 模型与对照模型的推荐结果，检验三类信息融合的必要性。具体地，本文设计了三种权重方案进行比较：

1. 仅保留共同好友数、Jaccard 系数和 Adamic-Adar 指数的结构模型；
2. 本文主模型，即融合结构、属性与行为信息的平衡模型；
3. 更强调用户画像匹配的画像强化模型。

三种方案得到的前 3 名候选结果如表 16 所示。

表 16: 第 2 问推荐模型对比结果

模型方案	第 1 名	第 2 名	第 3 名
结构模型	S2	S3	S9
主模型	S146	S24	S2
画像强化模型	S146	S34	S24

由表 16 可知，结构模型会显著偏向网络中的高连接候选，如 S2、S3、S9；而在加入属性与行为信息后，S146 与 S24 等更符合 S11 个人画像的候选会明显上升。主模型与结构模型的前 3 名重合度为 1/3，主模型与画像强化模型的前 3 名重合度为 2/3。这说明：

- 若只依赖网络结构，模型更容易推荐“二阶关系强”的中心节点；
- 当题目要求面向特定用户进行个性化推荐时，属性与行为信息会对排序产生实质性影响；
- 在引入画像信息后，S146 与 S24 始终位于前列，说明它们是相对稳定的主要候选；S2 则体现了结构桥接型推荐的作用。

因此，第 2 问的推荐结果不是偶然得到的，而是在结构关联和画像匹配之间取得平衡后的产物，具有较好的解释性和应用合理性。

进一步地，本文将 S11 的已有好友视为“已成边正例”，将随机非好友视为对照，以检验 SAB 得分是否能够将真实社交接近关系整体排到更高分区。具体做法是：保持第 2 问的特征构造方式和主模型权重不变，对除 S11 外的其余 199 名用户统一计算 SAB 得分，其中已有好友 37 名、非好友 162 名；随后从 162 名非好友中重复随机抽取 37 人，构造 2000 组等规模对照样本，结果如表 17 所示。

表 17：第 2 问“已有好友-随机非好友”SAB 对照检验结果

指标	结果
已有好友数 / 非好友数	37/162
随机对照方式	从 162 名非好友中等规模抽取 37 人，重复 2000 次
已有好友组 SAB 平均得分	0.375753
随机非好友对照组平均得分	0.256940
均值	
已有好友组 SAB 中位数	0.378462
随机非好友对照组中位数	0.237040
均值	
AUC	0.792626
达到好友组均值 / 中位数的	0/2000, 0/2000
随机样本数	

由表 17 可见，已有好友组的 SAB 平均得分和中位数都明显高于随机非好友对照，均值差达到 0.118813，中位数差达到 0.141422。同时，AUC 为 0.792626，说明若随机抽取一对“已有好友-非好友”用户，已有好友的 SAB 得分更高的概率约为 79.26%。更重要的是，在 2000 组随机对照中，没有任何一组的样本均值或中位数达到已有好友组水平，说明这种差异并非由偶然抽样造成。

该检验说明，同一套 SAB 特征与权重确实能够识别题目所关心的社交接近关系，而不是用于向 S11 推荐其现有好友。SAB 模型既能在非好友候选集中做个性化排序，也能在已有好友与随机非好友对照的口径下表现出合理的判别能力，因此将其作为第 2 问的主推荐模型是合适的。

9.3 第 3 问模型检验

本检验旨在验证第 3 问中关键用户识别与传播结论是否仅由单一权重设置或单组概率参数造成。为此，第 3 问从“权重方案对比”和“概率参数扰动”两方面检验模型稳健性。

首先，将第 3 问的关键用户预评分分别设置为“等权主方案”和“熵权对比方案”。其中，熵权法根据候选池中各指标的离散程度自动分配权重，得到的主要结果为：介数中心性 0.4623、发帖数 0.1546、度中心性 0.1298、互动频率 0.1153、 k -core 指标 0.1153，其余指标权重较小。两种方案得到的前 3 名结果如表 18 所示。

表 18: 第 3 问不同权重方案下的前 3 名关键用户

权重方案	第 1 名	第 2 名	第 3 名
等权主方案	S6	S3	S4
熵权对比方案	S6	S4	S12

由表 18 可见，两种权重方案的第 1 名都稳定为 S6，且前 3 名中有 2 个用户重合。这说明“S6 排名第 1”的判断较稳定；而 S3、S4、S12 的顺序差异，则反映了熵权法更偏重高离散度结构指标、等权法更强调多指标均衡的特点。考虑到本题是主题明确的校园传播问题，本文最终保留解释性更强的等权方案作为主结果。

其次，对转发概率公式中的正向系数与时间衰减系数进行 $\pm 10\%$ 扰动，构造“保守场景”和“乐观场景”，并考察最终关键用户 S6 的传播结果，如表 19 所示。

表 19: S6 在不同概率参数场景下的传播结果

场景	12h 平均覆盖	48h 平均覆盖	48h 平均转发
保守场景	158.93	196.82	80.11
基准场景	167.05	199.13	106.09
乐观场景	173.20	199.82	128.91

可以看到，即使在更保守的概率设定下，S6 在 48 小时内的平均覆盖人数仍达到 196.82；在更乐观的设定下，该值提升至 199.82。这说明参数扰动主要影响的是传播速度和转发规模，而不会改变 S6 排名第 1 的判断。此外，从误差分析角度看，第 3 问正文中给出的传播覆盖 95% Monte Carlo 置信区间实际上反映了随机仿真本身的估计误差：以 S6 为例，其 48 小时覆盖区间为 [196.00, 200.00]，区间宽度仅 4 人，表明 240 次独立仿真已能对传播规模给出较稳定的估计，不会因随机波动改变排序结论。

9.4 第 4 问模型检验

本检验旨在验证第 4 问优化策略的优势是否只是某一种启发式搜索偶然得到的结果，并说明该方案在相同约束下相对可解释基线仍有稳定优势。由于选做题的核心目标是最大化 48 小时传播范围，因此本文在相同候选池、相同两阶段配额、相同仿真次数和相同随机种子条件下，将优化策略与四类可解释基线进行比较，结果如表 20 所示。

表 20: 第 4 问优化策略与基线策略比较

策略	24h 平均覆盖	48h 平均覆盖	48h 区间
本文优化策略	196.78	199.80	[199, 200]
纯贪心策略	195.63	199.67	[198, 200]
最高度数策略	193.81	199.35	[198, 200]
最高活跃度策略	192.73	198.26	[197, 200]
最高文化参与度策略	196.69	199.78	[198, 200]

表 20 表明，本文优化策略在 24 小时与 48 小时两个关键指标上均取得最优结果。其中，相比“最高文化参与度”这一最强基线，本文方案在 48 小时平均覆盖上仍提升了 0.0167 人；相比“最高度数”和“最高活跃度”两类直观基线，则分别提升了 0.4548 人和 1.5405 人。由于全网规模只有 200 人，这一结果说明本文策略已在接近饱和的传播区间内取得了小幅而稳定的改进。

同时需要指出，第 4 问中各策略的 48 小时区间高度接近，反映出在当前数据规模下，文化类内容一旦完成首轮有效触达，后续扩散很容易逼近全网覆盖上限。因此，优化策略的优势不仅体现在最终覆盖人数上，也体现在更高的早期传播效率上：本文方案的 24 小时平均覆盖为 196.78，高于纯贪心策略的 195.63 和最高度数策略的 193.81。这说明先保证主题相关性、再补入跨社群节点的做法确实提升了前半程扩散速度。

十、 模型评价、改进与推广

10.1 模型优点

- 本文以校园社交网络图为基础，依次完成社群识别、好友推荐、关键用户识别和定向推送优化。四问之间前后衔接自然，前一问得到的社群结构与用户画像结果也能直接用于后一问求解。
- 各问指标都具有明确的现实含义。第 1 问的内部密度和跨社群联系对应圈内紧密程度与圈间连接强度，第 2 问的 SAB 得分同时反映网络接近程度、现实接触机会和后续互动可能性，第 3、第 4 问则将主题相关性、网络位置和活跃时段共同纳入传播分析，因此模型结论较易解释，也便于转化为平台运营建议。
- 模型检验较为完整。第 1 问比较了 Louvain 与 greedy modularity 的划分结果，并对功能重叠权重做了轻度扰动；第 2 问利用“已有好友-随机非好友”对照检验

了 SAB 得分的判别能力；第 3 问比较了等权与熵权方案，并进行了概率参数扰动；第 4 问又与多种可解释基线进行了对照，因此主要结论具有较好的可信度。

10.2 模型缺点

- 本文四问都建立在静态好友网络之上，传播过程也按单主题、单轮信息触达规则展开，没有显式刻画好友关系随时间变化、重复曝光以及多主题竞争等更复杂的平台场景。
- 受题目数据范围限制，第 2 问的特征权重、第 3 问和第 4 问中的部分候选阈值与辅助参数，仍主要依据解释性和可操作性进行设定，尚未利用更长时间的历史日志进行数据驱动标定。

10.3 模型改进

针对上述不足，后续可从以下两方面继续改进：

1. 若平台能够提供更长时间的好友关系和行为日志，可将静态网络扩展为动态图，并在传播仿真中加入重复曝光、内容竞争和阶段反馈更新机制，使第 3 问和第 4 问更贴近真实运营过程；
2. 若平台能够提供历史推荐接受记录、推送点击率和转发日志，可进一步用学习排序、A/B 测试或贝叶斯更新方法标定权重与阈值，把第 2 问的推荐评分和第 4 问的候选筛选从“可解释设定”推进到“数据驱动设定”。

10.4 模型推广

本文建立的“图建模 → 社群识别 → 个性化评分 → 传播仿真 → 约束优化”分析框架，不仅适用于本题给出的校园社交平台，也可以直接推广到校园通知推送、社团招新、竞赛组队、志愿活动动员等相关任务。具体应用时，只需把第 2 问至第 4 问中的主题变量替换为相应任务的相关度指标，并同步调整候选池阈值、推送名额和评价目标，便可形成面向不同运营场景的分析方案。

从更广的应用范围看，该框架也适用于企业内部社区、在线兴趣社区和公共服务平台。因为模型所需的核心数据主要是关系网络、基础属性和行为日志，这些数据在多数平台上都较易获取，所以模型既具有一定的可迁移性，也有实际应用的可能。

10.5 主要结论

本文围绕校园社交平台“好友关系与社交价值”分析问题，构建了从社群发现、好友推荐、关键用户识别到定向推送优化的一体化建模框架。第 1 问中，本文基于 Louvain 算法识别出 10 个社群，模块度为 0.247199，并进一步指出 C_4 在多个高密度社群之间起到了连接作用。第 2 问中，围绕用户 s11 构建了 SAB 可解释推荐模型，得到前 3 名推荐对象分别为 s146、s24 和 s2。第 3 问中，本文结合关键用户综合得分与离散传

播仿真，识别出 s6 在本文设定下具有最好的科技类话题传播表现，其在 48 小时内的平均覆盖人数达到 199.13，并在权重替换与概率扰动下保持稳定。

在选做题中，本文进一步将文化类话题的定向推送建模为带阶段配额约束的传播优化问题。基于 68 名活跃候选用户，本文先构造高相关初始解，再结合阶段贪心和局部交换修正完成求解，最终得到的优化方案在本文比较的方案中表现最好，在 48 小时内实现平均 199.80 人的传播覆盖。该结果说明，在接近全网覆盖上限的场景下，补入跨社群用户仍有作用，而早期扩散效率仍然是区分推送策略优劣的关键。

10.6 平台建议

基于本文的建模结果，可为校园社交平台提出如下建议：

1. 在好友推荐中同时考虑网络结构、属性相似度和行为相似度，避免仅凭共同好友数进行单一推荐；
2. 面向信息传播任务，应优先识别主题相关度高且具有社群桥梁作用的用户，而非只关注高度中心节点；
3. 在运营推送中，应结合用户活跃时段进行分阶段投放，尤其要重视下午时段和跨社群补点对早期扩散的促进作用；
4. 当推送预算有限时，建议先面向高相关活跃用户建立候选池，再通过仿真比较边际收益，以提升单位名额带来的传播回报。

若将这些建议进一步落到具体业务上，学术通知更适合优先触达学业相关社群与高信息密度用户，社团招新更适合起连接作用的节点与中午、下午活跃用户，文化活动策划推送则更适合优先选择文化参与度高且具备跨社群扩散能力的用户。这样的分类运营方式更容易与真实校园场景形成对应。

参考文献

- [1] 潘理, 吴鹏, 黄丹华. 在线社交网络群体发现研究进展[J]. 电子与信息学报, 2017, 39(09): 2097-2107.
- [2] 刘大有, 金弟, 何东晓, 等. 复杂网络社区挖掘综述[J]. 计算机研究与发展, 2013, 50(10): 2140-2154.
- [3] 张海涛, 周红磊, 张鑫蕊, 等. 在线社交网络的社区发现研究进展[J]. 图书情报工作, 2020, 64(09): 142-152.
- [4] FORTUNATO S. Community detection in graphs[J]. Physics Reports, 2010, 486(3-5): 75-174.

- [5] BLONDEL V D, GUILLAUME J L, LAMBIOTTE R, et al. Fast unfolding of communities in large networks[J]. Journal of Statistical Mechanics: Theory and Experiment, 2008, 2008(10): P10008.
- [6] 吕琳媛. 复杂网络链路预测[J]. 电子科技大学学报, 2010, 39(05): 651-661.
- [7] 张怡文, 岳丽华, 张义飞, 等. 基于共同用户和相似标签的好友推荐方法[J]. 计算机应用, 2013, 33(08): 2273-2275.
- [8] LÜ L, ZHOU T. Link prediction in complex networks: a survey[J]. Physica A: Statistical Mechanics and its Applications, 2011, 390(6): 1150-1170.
- [9] LEICHT E A, HOLME P, NEWMAN M E J. Vertex similarity in networks[J]. Physical Review E, 2006, 73(2): 026120.
- [10] LIBEN-NOWELL D, KLEINBERG J. The link-prediction problem for social networks[J]. Journal of the American Society for Information Science and Technology, 2007, 58(7): 1019-1031.
- [11] ADAMIC L A, ADAR E. Friends and neighbors on the web[J]. Social Networks, 2003, 25(3): 211-230.
- [12] 李翔. 复杂动态网络传播动力学[J]. 力学进展, 2008, 38(06): 723-732.
- [13] 韩忠明, 陈炎, 刘雯, 等. 社会网络节点影响力分析研究[J]. 软件学报, 2017, 28(01): 84-104.
- [14] KITSACK M, GALLOS L K, HAVLIN S, et al. Identification of influential spreaders in complex networks[J]. Nature Physics, 2010, 6(11): 888-893.
- [15] SEIDMAN S B. Network structure and minimum degree[J]. Social Networks, 1983, 5(3): 269-287.
- [16] KEMPE D, KLEINBERG J, TARDOS E. Maximizing the spread of influence through a social network[C]//Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2003: 137-146.
- [17] CHEN W, WANG Y, YANG S. Efficient influence maximization in social networks [C]//Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2009: 199-208.

附录

按照竞赛论文模板的常见写法，本文附录先给出支撑材料与程序文件列表，再补充模型总览流程图、详细求解过程和较繁杂结果表，最后列出各问核心算法代码片段，以增强论文的可复核性。本文全部脚本均由 Python 语言编写，结果文件集中保存在 `outputs/` 目录，程序脚本集中保存在 `scripts/` 目录。考虑到正文已经完整说明了建模思路和求解过程，附录只保留与结果复现直接相关的支撑材料和关键过程，不再铺陈全部中间细节。

本文所有建模、检验与可视化均基于题目给定的三张原始数据表完成，未额外引入网络公开数据，因此附录不再单列外部数据来源，以保证正文与附录的数据口径一致。

附录 1 支撑材料与结果文件列表

表 21: 支撑材料与结果文件说明表

材料目录	代表文件	主要说明
overview/	data_overview_table.csv、 network_summary.json	给出三张原始数据表整合后的数据概览与全网基础统计，是正文数据说明与建图参数的支撑材料
question1/	community_summary.csv、 community_relation_edges.csv、 community_overlap_similarity.csv、 question1_validation.json、*.png	对应第 1 问的社群划分结果、社群联系结果、功能重叠结果、检验结果与图形文件
question2/	s11_candidates.csv、s11_top3.json、 question2_validation.json、*.png	对应第 2 问的候选用户评分结果、前 3 名推荐结果、已有好友与随机非好友对照检验结果，以及相关图形文件
question3/	tech_seed_ranking.csv、 selected_seed_hourly_curve.csv、 baseline_comparison.csv、 question3_validation.json、*.png	对应第 3 问的关键用户排序、传播曲线、对比方案结果、稳健性检验结果与图形文件
question4/	candidate_pool.csv、 optimized_push_schedule.csv、 strategy_comparison.csv、 greedy_stage_log.csv、 local_search_log.csv、 question4_summary.json、*.png	对应第 4 问的候选池、最优推送方案、策略比较、阶段选人日志、局部交换日志与图形文件

附录 2 程序文件说明

表 22: 程序文件说明表

程序文件	对应问题	主要功能	主要输出
01_data_overview.py	数据预处理	统一三张数据表主键，构建无向好友网络，统计节点数、边数、连通性与基础网络指标	数据概览表、网络汇总信息
02_question1_community.py	第 1 问	执行 Louvain 社群识别，计算社群规模、内部密度、跨社群关系与功能重叠指标	社群划分表、社群画像表、关系矩阵
05_question1_assets.py	第 1 问	生成第 1 问图形，并完成多随机种子划分稳定性与功能重叠权重扰动检验	图形文件、检验结果 JSON
03_question2_recommendation.py	第 2 问	构造 S11 候选集，计算 SAB 综合得分，并完成“已有好友-随机非好友”对照检验	候选排序表、前 3 推荐说明、检验结果 JSON
06_question2_assets.py	第 2 问	生成前 10 名候选得分图和前 3 名贡献分解图	第 2 问图形文件
04_question3_seed_ranking.py	第 3 问	计算关键用户预评分，执行离散传播 Monte Carlo 仿真，输出关键用户排序、传播曲线与稳健性检验结果	前 10 关键用户表、传播曲线数据、检验结果 JSON
07_question3_assets.py	第 3 问	生成关键用户得分图、前 3 传播曲线图 and 最优种子传播图	第 3 问图形文件
08_question4_push_optimization.py	第 4 问	在配额约束下执行阶段贪心与局部交换优化，并输出策略比较与阶段日志	候选池、最优方案、策略比较表、运行日志
09_question4_assets.py	第 4 问	生成策略比较图、最优方案社群构成图和活跃时段画像图	第 4 问图形文件
common.py	公共模块	提供数据读取、字段清洗、建图和结果写出等公共函数，供各问脚本复用	公共函数支持

附录 3 四问模型总览流程图

为便于集中展示四个问题从输入数据到结果输出的整体链条，本附录补充给出四问模型总览图。与正文中的“建模思路”文字框架相比，这组图更适合快速对照各问的数据入口、核心模型和结果出口。

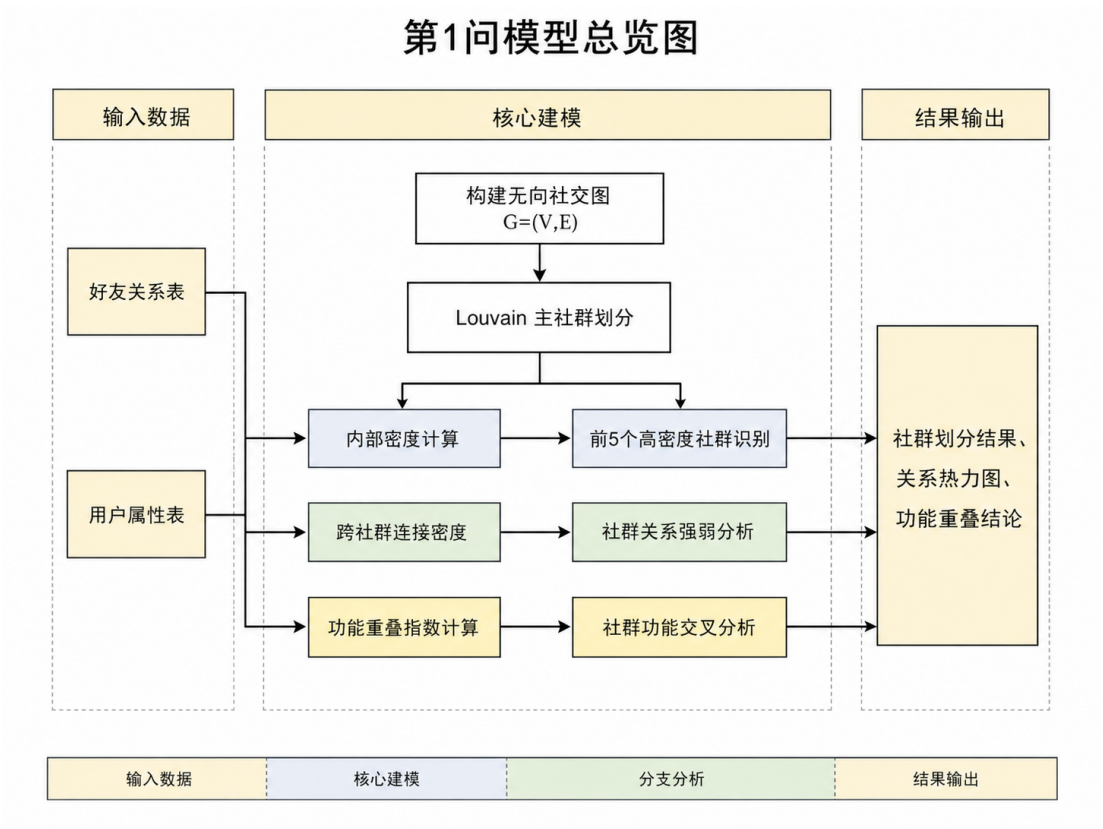


图 14: 第 1 问模型总览图

第2问模型总览图

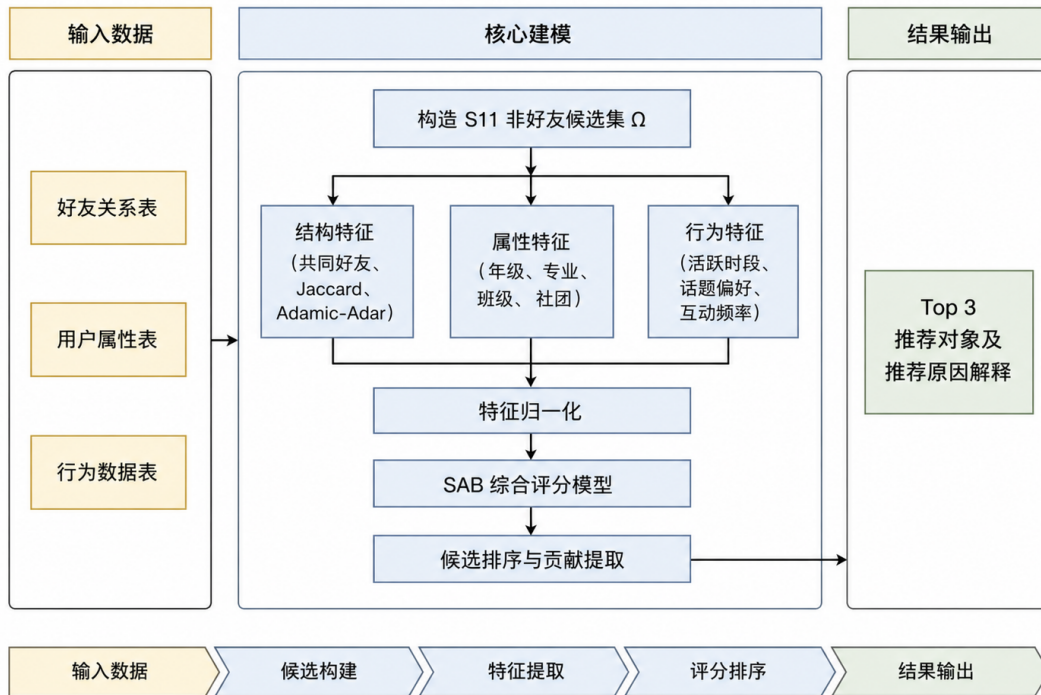


图 15: 第 2 问模型总览图

第3问模型总览图

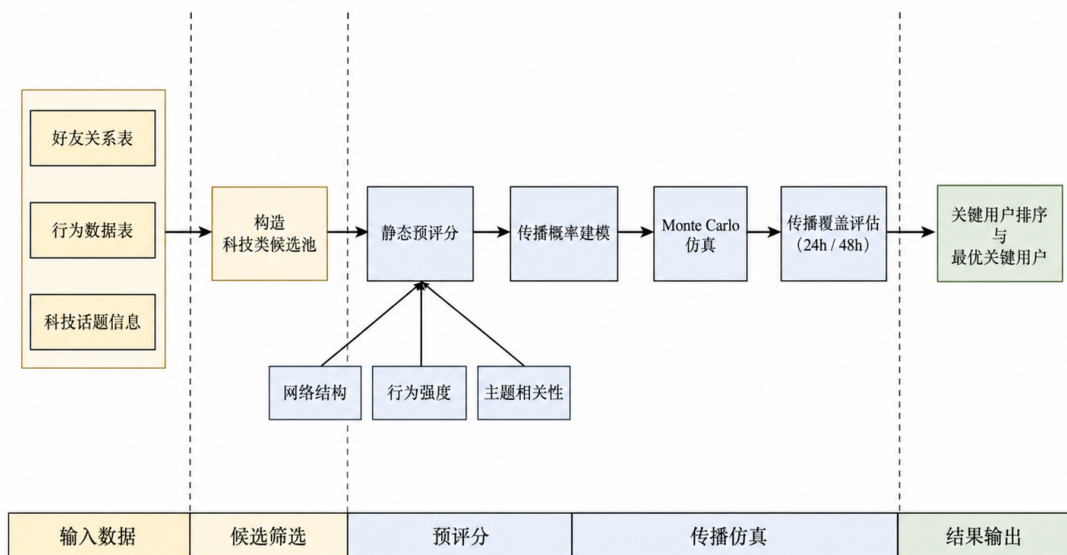


图 16: 第 3 问模型总览图

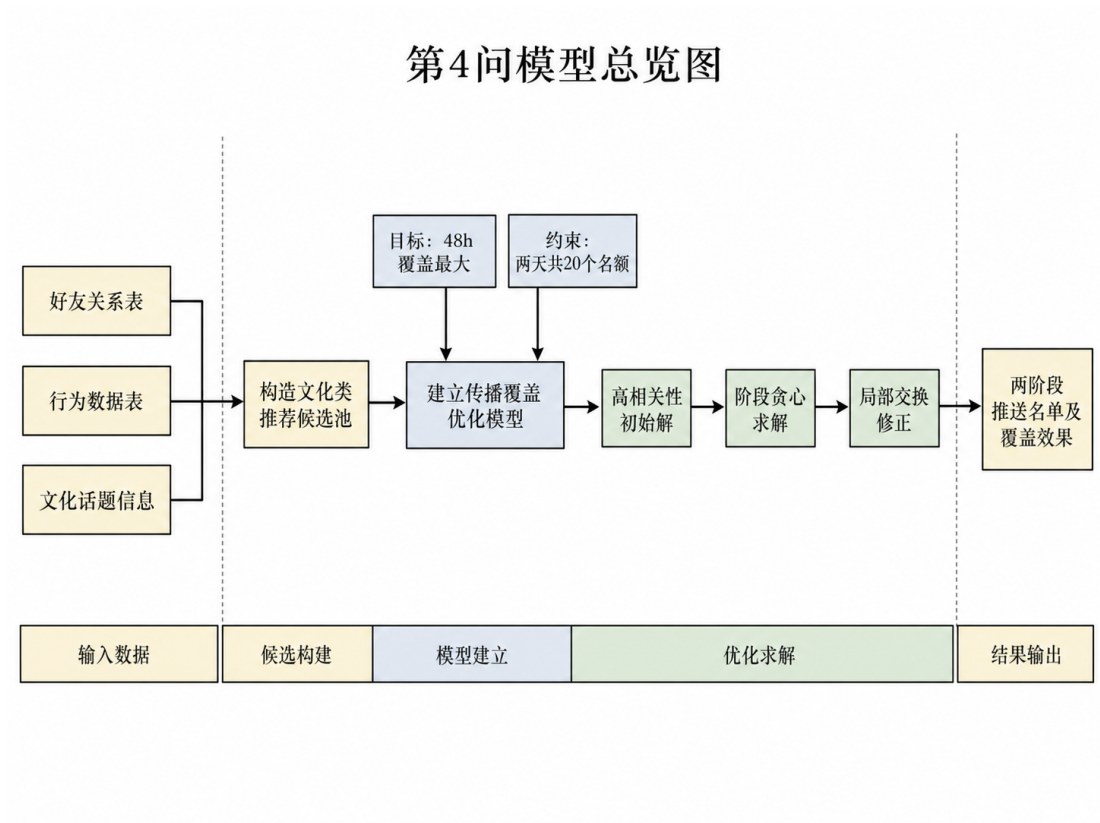


图 17: 第 4 问模型总览图

图 14 至图 17 依次对应第 1 至第 4 问。第 1 问从社交网络建图出发，完成社群识别、社群关系分析与功能重叠分析；第 2 问围绕用户 s11 构造候选集、提取结构属性行为特征，并进行 SAB 评分排序；第 3 问先筛选科技类候选池，再结合传播概率建模与 Monte Carlo 仿真识别关键用户；第 4 问则在文化类候选池基础上进行两阶段优化，输出两天推送名单及其覆盖结果。

附录 4 第 4 问两阶段推送优化的详细求解过程

正文已经给出第 4 问的优化目标、约束条件和核心流程图。为便于复核程序实现与结果输出之间的对应关系，这里把两阶段推送优化的求解过程再按执行顺序展开如下。

1. 候选池构建。基于好友关系、用户行为和文化话题信息，先筛出与文化类内容相关、且具备一定活跃度和扩散潜力的候选用户，形成初始候选池。
2. 预评分计算。对候选用户计算主题相关性、桥接作用、活跃时段匹配等指标，得到用于排序的预评分，为后续阶段选人提供统一比较尺度。
3. 初始解生成。在“每天最多推送 10 人、两天合计 20 人”的约束下，按天分别构造高相关性初始解，使第 1 天优先覆盖当前可迅速激活的用户，第 2 天补充尚未充分覆盖的社群与时段。
4. 阶段贪心求解。在固定日配额下，逐个加入能够带来最大边际覆盖增益的候选用户，并实时更新已覆盖节点状态，避免在同一社群和同一活跃时段上过度集中。
5. 局部交换修正。以贪心得到的方案为起点，对边际贡献偏低的用户尝试单点替换；若替换后 48 小时平均覆盖人数提高，则保留新的解。
6. 方案输出与比较。最终输出两天推送名单，并与“最高度数”“最高活跃度”“最高文化相关度”等基线策略比较，验证优化方案的有效性。

上述步骤分别对应候选池、推送名单、策略比较、阶段日志和局部交换日志等结果文件。正文保留了主要图表和关键结论，附录则将其中较长的名单与比较结果集中列出，便于对照程序输出进行复核。

附录 5 详细结果表

下面给出正文中未完全展开的若干结果表。这些表格主要用于支撑各问的结论判断，不改变正文已经给出的模型方案与结果口径。

表 23 汇总了全部 10 个社群的规模、内部密度、外连占比及主导画像。正文重点展示了高密度核心社群，附录保留完整摘要，便于横向比较各社群之间的结构差异。

表 23: 第 1 问全部社群的结构与画像摘要

社群编号	规模	内部密度	外连占比	主导年级	主导学科	主导专业	主导社团
1	10	0.3556	0.6832	研究生	工科	计算机科学	学生会
2	12	0.3485	0.6618	大四	理科	化学	电子科技协会
3	15	0.2571	0.6932	大一	文科	法学	机器人社团
4	31	0.2538	0.5881	大三	工科	机械工程	创新创业俱乐部
5	20	0.2368	0.6443	大三	理科	材料科学	电子科技协会
6	19	0.2222	0.6514	大四	理科	物理学	志愿者协会
7	14	0.2198	0.6296	大四	文科	法学	话剧社
8	18	0.2157	0.5901	大一	理科	机械工程	学生会
9	33	0.1591	0.5831	大二	文科	新闻传播	学生会
10	28	0.1402	0.6030	大二	文科	汉语言文学	摄影协会

表 24 给出 S11 的前 10 名推荐候选及其 SAB 得分构成。正文重点解释了前 3 名推荐对象，附录补充保留第 4 至第 10 名，便于比较“结构接近”“属性相似”和“行为匹配”三类因素在不同候选中的主导作用。

表 24: 第 2 问 S11 前 10 名推荐候选得分摘要

排名	候选用户	SAB 得分	共同好友数	共同社团数	话题相似度	结构贡献	属性贡献	行为贡献
1	S146	0.5717	5	0	0.90	0.1571	0.2000	0.2146
2	S24	0.5548	7	2	0.80	0.1890	0.2300	0.1358
3	S2	0.5504	14	0	0.55	0.3800	0.0000	0.1704
4	S34	0.5488	7	1	0.90	0.1967	0.1550	0.1971
5	S22	0.5210	7	0	0.85	0.2000	0.1300	0.1910
6	S149	0.5005	3	1	0.80	0.0947	0.2050	0.2008
7	S33	0.4714	5	0	0.85	0.1365	0.1300	0.2049
8	S9	0.4713	11	0	0.55	0.3008	0.0000	0.1705
9	S47	0.4706	7	0	0.85	0.2067	0.0800	0.1839
10	S19	0.4677	9	0	0.90	0.2565	0.0800	0.1312

表 25 对比了第 3 问中本文模型优选用户与两个典型基线用户的传播效果。正文主要保留了最优用户及其传播曲线，附录将对对比结果完整列出，以支撑“综合结构与行为信息比单指标排序更稳妥”的结论。

表 25: 第 3 问最优关键用户与基线用户对比

选择策略	用户编号	科技话题值	度数	12 小时覆盖均值	48 小时覆盖均值	48 小时转发均值	最终得分
本文模型优选用户	S6	8.0	59	166.8667	199.1125	105.7333	0.6673
主题相关度最高用户	S14	10.0	31	146.5708	196.8542	94.5000	0.6085
互动频率最高用户	S126	6.0	8	106.8667	155.9917	69.5833	0.4585

表 26 与表 27、表 28 分别给出第 4 问的策略比较结果，以及两天推送名单的详细构成。正文保留了比较图和核心解释，附录补充完整名单，便于检查各日推送对象的社群分布与时段分布是否符合模型设计。

表 26: 第 4 问不同推送策略的传播效果比较

策略名称	推送人数	12 小时覆盖均值	24 小时覆盖均值	48 小时覆盖均值	48 小时转发均值	与最优方案差距
精修优化方案	20	180.0738	196.7833	199.8000	114.6524	0.0000
贪心初始方案	20	179.5881	195.6286	199.6738	114.0548	0.1262
最高度数策略	20	177.9500	193.8143	199.3452	112.2500	0.4548
最高活跃度策略	20	176.8214	192.7310	198.2595	111.7167	1.5405
最高文化相关度策略	20	179.9929	196.6881	199.7833	114.3857	0.0167

表 27: 第 4 问第 1 天推送名单

次序	推送阶段	用户编号	社群编号	活跃时段	文化话题值	预评分
1	第 1 天	S127	7	下午	10.0	0.4109
2	第 1 天	S129	9	下午	10.0	0.3694
3	第 1 天	S162	10	晚上	10.0	0.4260
4	第 1 天	S184	8	下午	7.0	0.3675
5	第 1 天	S19	5	晚上	10.0	0.6167
6	第 1 天	S192	8	下午	10.0	0.3416
7	第 1 天	S27	5	晚上	8.0	0.5258
8	第 1 天	S32	10	下午	10.0	0.4693
9	第 1 天	S59	3	晚上	10.0	0.5078
10	第 1 天	S78	10	下午	10.0	0.3436

表 28: 第 4 问第 2 天推送名单

次序	推送阶段	用户编号	社群编号	活跃时段	文化话题值	预评分
11	第 2 天	S102	3	晚上	9.0	0.3410
12	第 2 天	S104	9	下午	10.0	0.3390
13	第 2 天	S11	3	下午	9.0	0.5827
14	第 2 天	S111	9	晚上	9.0	0.3868
15	第 2 天	S121	1	下午	9.0	0.3655
16	第 2 天	S131	7	下午	8.0	0.5278
17	第 2 天	S149	9	下午	9.0	0.3836
18	第 2 天	S160	8	全天	9.0	0.3683
19	第 2 天	S87	7	下午	9.0	0.4541
20	第 2 天	S94	4	下午	9.0	0.3703

附录 6 第 1 问核心算法代码

以下代码由 Python 语言编写，用于完成第 1 问的主社群识别。程序优先调用 Louvain 依赖包完成划分；若运行环境不具备该依赖，则退化为贪心模块度方法作为备选。

```
1 def detect_communities(graph: nx.Graph):
2     try:
3         from community import community_louvain # type: ignore
4
5         partition = community_louvain.best_partition(graph, random_state=42)
6         buckets: dict[int, set[str]] = {}
7         for node, cid in partition.items():
8             buckets.setdefault(cid, set()).add(node)
9         communities = list(buckets.values())
10        method = "louvain"
11    except Exception:
12        communities = list(nx.algorithms.community.greedy_modularity_communities(graph))
13        method = "greedy_modularity"
14    return communities, method
```

以下代码由 Python 语言编写，用于统计跨社群边数、跨社群连接密度，并基于年级、专业和社团分布计算功能重叠指数，是第 1 问社群关系分析的核心程序。

```
1 pair_counts: Counter[tuple[int, int]] = Counter()
2 for source, target in graph.edges():
3     cid_u = node_to_community[source]
4     cid_v = node_to_community[target]
5     if cid_u == cid_v:
6         continue
7     pair = tuple(sorted((cid_u, cid_v)))
8     pair_counts[pair] += 1
9
10 relation_rows = []
11 for (cid_u, cid_v), edge_count in sorted(pair_counts.items(), key=lambda item: (-item[1], item[0])):
12     :
13     size_u = rows[cid_u - 1]["size"]
14     size_v = rows[cid_v - 1]["size"]
15     relation_rows.append(
16         {
17             "source_community": cid_u,
18             "target_community": cid_v,
19             "cross_edges": edge_count,
20             "cross_density": round(edge_count / (size_u * size_v), 6),
21         }
22     )
23
24 overlap_rows = []
25 for profile_a, profile_b in combinations(raw_profiles, 2):
26     grade_similarity = cosine_similarity(profile_a["grade_vector"], profile_b["grade_vector"])
27     major_similarity = cosine_similarity(profile_a["major_vector"], profile_b["major_vector"])
28     club_similarity = cosine_similarity(profile_a["club_vector"], profile_b["club_vector"])
29     overall_similarity = round((grade_similarity + major_similarity + club_similarity) / 3, 6)
30     overlap_rows.append(
```

```

30         {
31             "community_a": profile_a["community_id"],
32             "community_b": profile_b["community_id"],
33             "grade_similarity": grade_similarity,
34             "major_similarity": major_similarity,
35             "club_similarity": club_similarity,
36             "overall_similarity": overall_similarity,
37             "both_in_top5": int(
38                 profile_a["community_id"] <= 5 and profile_b["community_id"] <= 5
39             ),
40         }
41     )
42     overlap_rows.sort(
43         key=lambda item: (-item["both_in_top5"], -item["overall_similarity"], item["community_a"], item
44             ["community_b"])
45     )
46     membership_rows = [
47         {"student_id": node, "community_id": community_id}
48         for node, community_id in sorted(node_to_community.items())
49     ]
50
51     write_csv(OUTPUT_DIR / "question1" / "community_summary.csv", rows)
52     write_csv(OUTPUT_DIR / "question1" / "community_relation_edges.csv", relation_rows)
53     write_csv(OUTPUT_DIR / "question1" / "community_membership.csv", membership_rows)
54     write_csv(OUTPUT_DIR / "question1" / "community_overlap_similarity.csv", overlap_rows)
55     write_json(OUTPUT_DIR / "question1" / "community_profiles.json", detailed_profiles)
56     write_json(
57         OUTPUT_DIR / "question1" / "community_meta.json",
58         {
59             "method": method,
60             "community_count": len(raw_profiles),
61             "modularity": modularity,
62             "top5_by_density": detailed_profiles[:5],
63             "strongest_pairs": relation_rows[:8],
64             "top5_functional_overlap": [
65                 row for row in overlap_rows if row["both_in_top5"] == 1
66             ][:8],
67         },

```

附录 7 第 2 问核心算法代码

以下代码由 Python 语言编写，实现第 2 问 SAB 综合评分模型中的话题相似度计算、特征归一化取值和加权求和过程，是推荐排序的核心评分程序。

```

1 def compute_topic_similarity(target_behavior: dict[str, object], behavior: dict[str, object]) -> float:
2     # 题目只给出了话题参与次数，因此这里把科技/文化话题差异压缩为
3     # [0, 1] 区间内的单一相似度分值。
4     tech_gap = abs(float(target_behavior.get("tech_topic", 0.0)) - float(behavior.get("tech_topic",
5         0.0)))
6     culture_gap = abs(float(target_behavior.get("culture_topic", 0.0)) - float(behavior.get("
7         culture_topic", 0.0)))
8     return max(0.0, 1.0 - (tech_gap + culture_gap) / 20.0)

```

```

9 def feature_value(
10     row: dict[str, object],
11     feature: str,
12     normalized_maps: dict[str, dict[str, float]],
13 ) -> float:
14     if feature in normalized_maps:
15         return normalized_maps[feature][str(row["candidate_id"])]
16     return float(row[feature])
17
18
19 def compute_scores(
20     base_rows: list[dict[str, object]],
21     weights: dict[str, float],
22     normalized_maps: dict[str, dict[str, float]],
23 ) -> list[dict[str, object]]:
24     # 不同推荐版本共用同一候选集和归一化特征，变化的只有验证时采用
25     # 的权重方案。
26     scored_rows: list[dict[str, object]] = []
27     for base_row in base_rows:
28         row = dict(base_row)
29         contributions: dict[str, float] = {}
30         normalized_values: dict[str, float] = {}
31         total_score = 0.0
32
33         for feature, weight in weights.items():
34             value = feature_value(row, feature, normalized_maps)
35             normalized_values[feature] = round(value, 6)
36             contribution = round(weight * value, 6)
37             contributions[feature] = contribution
38             total_score += contribution
39
40         support_features = [
41             FEATURE_LABELS[feature]
42             for feature, contribution in sorted(contributions.items(), key=lambda item: (-item[1], item
43 [0]))
44             if contribution > 0
45         ][:4]
46
47         row["score"] = round(total_score, 6)
48         row["top_support_features"] = " | ".join(support_features)
49         for feature, value in normalized_values.items():
50             row[f"{feature}_norm"] = value
51         for feature, contribution in contributions.items():
52             row[f"{feature}_contribution"] = contribution
53
54         row["structure_contribution"] = round(
55             sum(contributions.get(feature, 0.0) for feature in STRUCTURE_FEATURES), 6
56         )
57         row["attribute_contribution"] = round(
58             sum(contributions.get(feature, 0.0) for feature in ATTRIBUTE_FEATURES), 6
59         )
60         row["behavior_contribution"] = round(
61             sum(contributions.get(feature, 0.0) for feature in BEHAVIOR_FEATURES), 6
62         )
63         scored_rows.append(row)

```

```

64     scored_rows.sort(key=lambda item: (-float(item["score"]), item["candidate_id"]))
65     for rank, row in enumerate(scored_rows, start=1):
66         row["rank"] = rank
67     return scored_rows

```

以下代码由 Python 语言编写，用于构造 S11 推荐问题的候选集，并提取结构、属性和行为三类特征，是第 2 问输入层与特征层的核心程序。

```

1  def build_base_rows(
2      graph: nx.Graph,
3      attr_map: dict[str, dict[str, object]],
4      behavior_map: dict[str, dict[str, object]],
5      community_map: dict[str, int],
6      relation_density_map: dict[tuple[int, int], float],
7      relation_edge_map: dict[tuple[int, int], int],
8      include_existing_friends: bool,
9  ) -> tuple[list[dict[str, object]], dict[str, dict[str, float]], set[str], dict[str, object], dict[str,
    object], int | None]:
10     target_neighbors = set(graph.neighbors(TARGET))
11     target_attr = attr_map.get(TARGET, {})
12     target_behavior = behavior_map.get(TARGET, {})
13     target_community = community_map.get(TARGET)
14
15     base_rows: list[dict[str, object]] = []
16     raw_maps = {
17         "common_neighbors": {},
18         "jaccard": {},
19         "adamic_adar": {},
20         "shared_clubs": {},
21         "interaction_similarity": {},
22         "activity_similarity": {},
23         "posts_similarity": {},
24     }
25
26     for node in graph.nodes:
27         if node == TARGET:
28             continue
29         if not include_existing_friends and node in target_neighbors:
30             continue
31
32         common_neighbors = len(list(nx.common_neighbors(graph, TARGET, node)))
33         union = len(target_neighbors | set(graph.neighbors(node)))
34         jaccard = common_neighbors / union if union else 0.0
35         adamic_adar = 0.0
36         for _, _, score in nx.adamic_adar_index(graph, [(TARGET, node)]):
37             adamic_adar = score
38
39         attr = attr_map.get(node, {})
40         behavior = behavior_map.get(node, {})
41         target_clubs = set(target_attr.get("clubs", []))
42         candidate_clubs = set(attr.get("clubs", []))
43         shared_club_names = sorted(target_clubs & candidate_clubs)
44         shared_clubs = len(shared_club_names)
45         same_major_category = int(
46             target_attr.get("major_category") == attr.get("major_category")

```

```

47         and target_attr.get("major_category") is not None
48     )
49     same_major = int(target_attr.get("major") == attr.get("major") and target_attr.get("major") is
not None)
50     same_class = int(
51         target_attr.get("class_name") == attr.get("class_name")
52         and target_attr.get("class_name") is not None
53     )
54     same_grade = int(target_attr.get("grade") == attr.get("grade") and target_attr.get("grade") is
not None)
55     same_active_period = int(
56         target_behavior.get("active_period") == behavior.get("active_period")
57         and target_behavior.get("active_period") is not None
58     )
59     topic_similarity = compute_topic_similarity(target_behavior, behavior)
60     interaction_gap = abs(
61         float(target_behavior.get("interaction_freq", 0.0)) - float(behavior.get("interaction_freq"
, 0.0))
62     )
63     activity_gap = abs(
64         float(target_behavior.get("activity_count", 0.0)) - float(behavior.get("activity_count",
0.0))
65     )
66     posts_gap = abs(float(target_behavior.get("posts", 0.0)) - float(behavior.get("posts", 0.0)))
67
68     candidate_community = community_map.get(node)
69     same_community = int(candidate_community == target_community)
70     community_relation_density = 0.0
71     community_cross_edges = 0
72     if (
73         candidate_community is not None
74         and target_community is not None
75         and candidate_community != target_community
76     ):
77         pair = pair_key(target_community, candidate_community)
78         community_relation_density = relation_density_map.get(pair, 0.0)
79         community_cross_edges = relation_edge_map.get(pair, 0)
80
81     row = {
82         "candidate_id": node,
83         "is_existing_friend": int(node in target_neighbors),
84         "candidate_community": candidate_community or 0,
85         "same_community": same_community,
86         "community_relation_density": round(community_relation_density, 6),
87         "community_cross_edges": community_cross_edges,
88         "candidate_grade": attr.get("grade", ""),
89         "candidate_major_category": attr.get("major_category", ""),
90         "candidate_major": attr.get("major", ""),
91         "candidate_class_name": attr.get("class_name", ""),
92         "candidate_clubs": " | ".join(attr.get("clubs", [])),
93         "candidate_active_period": behavior.get("active_period", ""),
94         "common_neighbors": common_neighbors,
95         "jaccard": round(jaccard, 6),
96         "adamic_adar": round(adamic_adar, 6),
97         "shared_clubs": shared_clubs,
98         "shared_club_names": " | ".join(shared_club_names),

```



```

99         "same_major_category": same_major_category,
100         "same_major": same_major,
101         "same_class": same_class,
102         "same_grade": same_grade,
103         "same_active_period": same_active_period,
104         "topic_similarity": round(topic_similarity, 6),
105         "interaction_gap": round(interaction_gap, 6),
106         "activity_gap": round(activity_gap, 6),
107         "posts_gap": round(posts_gap, 6),
108     }
109     base_rows.append(row)
110
111     raw_maps["common_neighbors"][node] = common_neighbors
112     raw_maps["jaccard"][node] = jaccard
113     raw_maps["adamic_adar"][node] = adamic_adar
114     raw_maps["shared_clubs"][node] = shared_clubs
115     raw_maps["interaction_similarity"][node] = -interaction_gap
116     raw_maps["activity_similarity"][node] = -activity_gap
117     raw_maps["posts_similarity"][node] = -posts_gap
118
119     return base_rows, raw_maps, target_neighbors, target_attr, target_behavior, target_community

```

以下代码由 Python 语言编写，用于将已有好友与随机非好友构造成对照组，检验 SAB 得分能否识别真实社交接近关系，对应正文第 2 问的模型检验部分。

```

1  def validate_existing_friends_vs_random_nonfriends(
2      graph: nx.Graph,
3      attr_map: dict[str, dict[str, object]],
4      behavior_map: dict[str, dict[str, object]],
5      community_map: dict[str, int],
6      relation_density_map: dict[tuple[int, int], float],
7      relation_edge_map: dict[tuple[int, int], int],
8  ) -> dict[str, object]:
9      # 将已有好友视作“已成边正例”，将随机非好友视作对照，检验 SAB
10     # 是否能识别真实存在的社交接近关系。
11     base_rows, raw_maps, _, _, _, _ = build_base_rows(
12         graph,
13         attr_map,
14         behavior_map,
15         community_map,
16         relation_density_map,
17         relation_edge_map,
18         include_existing_friends=True,
19     )
20     normalized_maps = {feature: normalize(values) for feature, values in raw_maps.items()}
21     scored_rows = compute_scores(base_rows, MAIN_WEIGHTS, normalized_maps)
22
23     friend_rows = [row for row in scored_rows if int(row["is_existing_friend"]) == 1]
24     nonfriend_rows = [row for row in scored_rows if int(row["is_existing_friend"]) == 0]
25     friend_scores = [float(row["score"]) for row in friend_rows]
26     nonfriend_scores = [float(row["score"]) for row in nonfriend_rows]
27
28     rng = random.Random(42)
29     sample_count = 2000
30     random_sample_means: list[float] = []

```

```

31 random_sample_medians: list[float] = []
32 for _ in range(sample_count):
33     sample_rows = rng.sample(nonfriend_rows, len(friend_rows))
34     sample_scores = [float(row["score"]) for row in sample_rows]
35     random_sample_means.append(statistics.mean(sample_scores))
36     random_sample_medians.append(statistics.median(sample_scores))
37
38 friend_mean = statistics.mean(friend_scores)
39 friend_median = statistics.median(friend_scores)
40 random_mean_avg = statistics.mean(random_sample_means)
41 random_median_avg = statistics.mean(random_sample_medians)
42 mean_not_lower_count = sum(sample_mean >= friend_mean for sample_mean in random_sample_means)
43 median_not_lower_count = sum(
44     sample_median >= friend_median for sample_median in random_sample_medians
45 )
46
47 return {
48     "friend_count": len(friend_rows),
49     "nonfriend_count": len(nonfriend_rows),
50     "resample_count": sample_count,
51     "friend_score_mean": round(friend_mean, 6),
52     "friend_score_median": round(friend_median, 6),
53     "nonfriend_score_mean": round(statistics.mean(nonfriend_scores), 6),
54     "nonfriend_score_median": round(statistics.median(nonfriend_scores), 6),
55     "random_nonfriend_sample_mean_avg": round(random_mean_avg, 6),
56     "random_nonfriend_sample_median_avg": round(random_median_avg, 6),
57     "mean_gap_vs_random_samples": round(friend_mean - random_mean_avg, 6),
58     "median_gap_vs_random_samples": round(friend_median - random_median_avg, 6),
59     "auc_friend_vs_nonfriend": auc_from_scores(friend_scores, nonfriend_scores),
60     "random_sample_mean_ge_friend_mean_count": mean_not_lower_count,
61     "random_sample_median_ge_friend_median_count": median_not_lower_count,
62     "top10_friend_count": sum(int(row["is_existing_friend"]) for row in scored_rows[:10]),
63     "top20_friend_count": sum(int(row["is_existing_friend"]) for row in scored_rows[:20]),
64 }

```

附录 8 第 3 问核心算法代码

以下代码由 Python 语言编写，实现离散传播仿真中的时序逻辑和题设给出的 Logistic 转发概率公式，是第 3 问传播机制建模的基础程序。

```

1 def next_active_time(active_period: str, current_time: float) -> float:
2     # 将用户活跃时段换算为“在当前事件之后何时看到内容”，时间
3     # 均以中午 12:00 发帖时刻作为零点。
4     if active_period == ALL_DAY:
5         return current_time
6
7     target_hour = PERIOD_HOUR[active_period]
8     absolute_hour = 12.0 + current_time
9     day_index = math.floor(absolute_hour / 24.0)
10    candidate = day_index * 24.0 + target_hour
11    if candidate < absolute_hour - 1e-9:
12        candidate += 24.0
13    return candidate - 12.0
14
15

```

```

16 def share_probability(
17     topic_participation: float,
18     interaction_freq: float,
19     delay_hours: float,
20     scenario: ProbabilityScenario,
21 ) -> float:
22     # 直接实现题目给出的 Logistic 转发概率公式；其中情景缩放只用于
23     # 敏感性分析，不改变主模型的基本结构。
24     z_value = (
25         -3.5
26         + 3.0 * scenario.topic_scale * (topic_participation / 10.0)
27         + 2.0 * scenario.interaction_scale * (interaction_freq / 25.0)
28         - 0.08 * scenario.decay_scale * delay_hours
29     )
30     return 1.0 / (1.0 + math.exp(-z_value))

```

以下代码由 Python 语言编写，实现事件队列式传播仿真过程，并在题目要求的关键时间节点上汇总覆盖均值与区间结果。

```

1 def simulate_once(
2     graph: nx.Graph,
3     behavior_map: dict[str, dict[str, float | str]],
4     seed_user: str,
5     rng: random.Random,
6     scenario: ProbabilityScenario,
7 ) -> tuple[dict[str, float], dict[str, float]]:
8     # 采用事件队列而不是固定时间步推进，使每次曝光都发生在接收者
9     # 的下一个活跃时刻。
10    seen_time = {seed_user: 0.0}
11    shared_time = {seed_user: 0.0}
12    event_queue: list[tuple[float, int, str, str]] = []
13    event_counter = 0
14
15    for friend in graph.neighbors(seed_user):
16        view_time = next_active_time(str(behavior_map[friend]["active_period"]), 0.0)
17        if view_time <= HORIZON_HOURS + 1e-9:
18            heapq.heappush(event_queue, (view_time, event_counter, seed_user, friend))
19            event_counter += 1
20            if view_time < seen_time.get(friend, float("inf")):
21                seen_time[friend] = view_time
22
23    while event_queue:
24        event_time, _, _, target_user = heapq.heappop(event_queue)
25        if event_time > HORIZON_HOURS + 1e-9:
26            break
27        if target_user in shared_time:
28            continue
29
30        behavior = behavior_map[target_user]
31        probability = share_probability(
32            topic_participation=float(behavior["tech_topic"]),
33            interaction_freq=float(behavior["interaction_freq"]),
34            delay_hours=event_time,
35            scenario=scenario,
36        )

```

```

37         if rng.random() >= probability:
38             continue
39
40         shared_time[target_user] = event_time
41         for friend in graph.neighbors(target_user):
42             if friend in shared_time:
43                 continue
44             next_view_time = next_active_time(str(behavior_map[friend]["active_period"]), event_time)
45             if next_view_time <= HORIZON_HOURS + 1e-9:
46                 heapq.heappush(event_queue, (next_view_time, event_counter, target_user, friend))
47                 event_counter += 1
48                 if next_view_time < seen_time.get(friend, float("inf")):
49                     seen_time[friend] = next_view_time
50
51     return seen_time, shared_time
52
53
54 def summarize_simulation(
55     graph: nx.Graph,
56     behavior_map: dict[str, dict[str, float | str]],
57     seed_user: str,
58     runs: int,
59     scenario: ProbabilityScenario,
60     checkpoints: list[float],
61 ) -> dict[str, object]:
62     # 对随机传播重复仿真数百次，并在题目要求的时间节点汇总均值与
63     # Monte Carlo 区间结果。
64     seen_runs: dict[float, list[int]] = {checkpoint: [] for checkpoint in checkpoints}
65     shared_runs: dict[float, list[int]] = {checkpoint: [] for checkpoint in checkpoints}
66
67     base_seed = 20260430 + int(seed_user[1:])
68     for run_index in range(runs):
69         rng = random.Random(base_seed + run_index)
70         seen_time, shared_time = simulate_once(graph, behavior_map, seed_user, rng, scenario)
71         seen_counts = count_by_checkpoints(seen_time, checkpoints)
72         shared_counts = count_by_checkpoints(shared_time, checkpoints)
73         for checkpoint in checkpoints:
74             seen_runs[checkpoint].append(seen_counts[checkpoint])
75             shared_runs[checkpoint].append(shared_counts[checkpoint])
76
77     seen_summary = {
78         f"{int(checkpoint)}h": {
79             "mean": round(statistics.mean(values), 4),
80             "std": round(statistics.pstdev(values), 4),
81             "p025": round(empirical_quantile(values, INTERVAL_LOW), 4),
82             "p975": round(empirical_quantile(values, INTERVAL_HIGH), 4),
83         }
84         for checkpoint, values in seen_runs.items()
85     }
86     shared_summary = {
87         f"{int(checkpoint)}h": {
88             "mean": round(statistics.mean(values), 4),
89             "std": round(statistics.pstdev(values), 4),
90             "p025": round(empirical_quantile(values, INTERVAL_LOW), 4),
91             "p975": round(empirical_quantile(values, INTERVAL_HIGH), 4),
92         }

```

```

93     for checkpoint, values in shared_runs.items()
94 }
95 return {
96     "seed_user": seed_user,
97     "runs": runs,
98     "scenario": scenario.name,
99     "seen": seen_summary,
100     "shared": shared_summary,
101 }

```

以下代码由 Python 语言编写，用于构造关键用户结构-行为特征，并将静态种子得分与动态传播表现融合为最终排序结果。

```

1 def build_feature_rows(
2     graph: nx.Graph,
3     behavior_map: dict[str, dict[str, float | str]],
4 ) -> dict[str, dict[str, float | str]]:
5     # 这一层预评分先刻画“主题契合度 + 网络位置”，再与后续传播
6     # 表现结合，从而保持最终排序的可解释性。
7     degree centrality = nx.degree_centrality(graph)
8     betweenness = nx.betweenness_centrality(graph)
9     closeness = nx.closeness_centrality(graph)
10    core_number_raw = nx.core_number(graph)
11
12    interaction_norm = normalize({user: float(values["interaction_freq"]) for user, values in
13        behavior_map.items()})
14    posts_norm = normalize({user: float(values["posts"]) for user, values in behavior_map.items()})
15    tech_norm = normalize({user: float(values["tech_topic"]) for user, values in behavior_map.items()})
16    core_norm = normalize({user: float(core) for user, core in core_number_raw.items()})
17
18    audience_timing_score: dict[str, float] = {}
19    for user in graph.nodes:
20        neighbors = list(graph.neighbors(user))
21        if not neighbors:
22            audience_timing_score[user] = 0.0
23            continue
24        delay_scores = [
25            1.0 - PERIOD_DELAY_FROM_NOON[str(behavior_map[neighbor]["active_period"])] / 22.0
26            for neighbor in neighbors
27        ]
28        audience_timing_score[user] = round(sum(delay_scores) / len(delay_scores), 6)
29
30    rows: dict[str, dict[str, float | str]] = {}
31    for user in graph.nodes:
32        behavior = behavior_map[user]
33        rows[user] = {
34            "user_id": user,
35            "degree": graph.degree(user),
36            "degree_centrality": round(degree_centrality[user], 6),
37            "betweenness": round(betweenness[user], 6),
38            "closeness": round(closeness[user], 6),
39            "core_number": core_number_raw[user],
40            "core_number_norm": round(core_norm[user], 6),
41            "interaction_freq": float(behavior["interaction_freq"]),
42            "interaction_freq_norm": round(interaction_norm[user], 6),

```

```

42         "posts": float(behavior["posts"]),
43         "posts_norm": round(posts_norm[user], 6),
44         "activity_count": float(behavior["activity_count"]),
45         "tech_topic": float(behavior["tech_topic"]),
46         "tech_topic_norm": round(tech_norm[user], 6),
47         "active_period": str(behavior["active_period"]),
48         "audience_timing_score": audience_timing_score[user],
49     }
50     return rows
51
52
53 def weighted_seed_score(row: dict[str, float | str], weights: dict[str, float]) -> float:
54     return round(sum(weights[key] * float(row[key]) for key in weights), 6)
55
56
57 def simulation_score(simulation_summary: dict[str, object], total_nodes: int) -> float:
58     seen = simulation_summary["seen"]
59     shared = simulation_summary["shared"]
60     score = (
61         SIMULATION_WEIGHTS["seen_6h"] * float(seen["6h"]["mean"]) / total_nodes
62         + SIMULATION_WEIGHTS["seen_12h"] * float(seen["12h"]["mean"]) / total_nodes
63         + SIMULATION_WEIGHTS["seen_24h"] * float(seen["24h"]["mean"]) / total_nodes
64         + SIMULATION_WEIGHTS["seen_48h"] * float(seen["48h"]["mean"]) / total_nodes
65         + SIMULATION_WEIGHTS["shared_48h"] * float(shared["48h"]["mean"]) / total_nodes
66     )
67     return round(score, 6)
68
69
70 def build_ranking_rows(
71     feature_rows: dict[str, dict[str, float | str]],
72     simulation_rows: dict[str, dict[str, object]],
73     candidate_users: list[str],
74     total_nodes: int,
75     main_weights: dict[str, float],
76     entropy_weights: dict[str, float],
77 ) -> list[dict[str, object]]:
78     # 最终排序把静态种子得分与动态传播表现结合起来，避免高中心性但
79     # 低主题相关的用户仅凭度数优势主导结果。
80     ranking_rows: list[dict[str, object]] = []
81     for user in candidate_users:
82         feature_row = dict(feature_rows[user])
83         summary = simulation_rows[user]
84         seed_score_main = weighted_seed_score(feature_row, main_weights)
85         seed_score_entropy = weighted_seed_score(feature_row, entropy_weights)
86
87         sim_score = simulation_score(summary, total_nodes)
88         final_score = round(0.35 * seed_score_main + 0.65 * sim_score, 6)
89         final_score_entropy = round(0.35 * seed_score_entropy + 0.65 * sim_score, 6)
90
91         ranking_rows.append(
92             {
93                 **feature_row,
94                 "candidate_pool_flag": int(float(feature_row["tech_topic"]) >= TOPIC_THRESHOLD),
95                 "seed_score_main": seed_score_main,
96                 "seed_score_entropy": seed_score_entropy,
97                 "simulation_score": sim_score,

```

```

98         "final_score": final_score,
99         "final_score_entropy": final_score_entropy,
100         "mean_seen_6h": float(summary["seen"]["6h"]["mean"]),
101         "std_seen_6h": float(summary["seen"]["6h"]["std"]),
102         "p025_seen_6h": float(summary["seen"]["6h"]["p025"]),
103         "p975_seen_6h": float(summary["seen"]["6h"]["p975"]),
104         "mean_seen_12h": float(summary["seen"]["12h"]["mean"]),
105         "std_seen_12h": float(summary["seen"]["12h"]["std"]),
106         "p025_seen_12h": float(summary["seen"]["12h"]["p025"]),
107         "p975_seen_12h": float(summary["seen"]["12h"]["p975"]),
108         "mean_seen_24h": float(summary["seen"]["24h"]["mean"]),
109         "std_seen_24h": float(summary["seen"]["24h"]["std"]),
110         "p025_seen_24h": float(summary["seen"]["24h"]["p025"]),
111         "p975_seen_24h": float(summary["seen"]["24h"]["p975"]),
112         "mean_seen_48h": float(summary["seen"]["48h"]["mean"]),
113         "std_seen_48h": float(summary["seen"]["48h"]["std"]),
114         "p025_seen_48h": float(summary["seen"]["48h"]["p025"]),
115         "p975_seen_48h": float(summary["seen"]["48h"]["p975"]),
116         "mean_shared_6h": float(summary["shared"]["6h"]["mean"]),
117         "std_shared_6h": float(summary["shared"]["6h"]["std"]),
118         "p025_shared_6h": float(summary["shared"]["6h"]["p025"]),
119         "p975_shared_6h": float(summary["shared"]["6h"]["p975"]),
120         "mean_shared_12h": float(summary["shared"]["12h"]["mean"]),
121         "std_shared_12h": float(summary["shared"]["12h"]["std"]),
122         "p025_shared_12h": float(summary["shared"]["12h"]["p025"]),
123         "p975_shared_12h": float(summary["shared"]["12h"]["p975"]),
124         "mean_shared_24h": float(summary["shared"]["24h"]["mean"]),
125         "std_shared_24h": float(summary["shared"]["24h"]["std"]),
126         "p025_shared_24h": float(summary["shared"]["24h"]["p025"]),
127         "p975_shared_24h": float(summary["shared"]["24h"]["p975"]),
128         "mean_shared_48h": float(summary["shared"]["48h"]["mean"]),
129         "std_shared_48h": float(summary["shared"]["48h"]["std"]),
130         "p025_shared_48h": float(summary["shared"]["48h"]["p025"]),
131         "p975_shared_48h": float(summary["shared"]["48h"]["p975"]),
132     }
133 )
134
135 ranking_rows.sort(key=lambda item: (-float(item["final_score"]), -float(item["mean_seen_48h"]),
136                                     item["user_id"]))
137 for rank, row in enumerate(ranking_rows, start=1):
138     row["rank"] = rank
139 return ranking_rows

```

附录 9 第 4 问核心算法代码

以下代码由 Python 语言编写，用于构造文化类推送候选池，并计算后续优化所需的预评分指标，是第 4 问候选筛选阶段的核心程序。

```

1 def build_candidate_rows(
2     graph: nx.Graph,
3     attr_map: dict[str, dict[str, object]],
4     behavior_map: dict[str, dict[str, float | str]],
5     community_map: dict[str, int],
6 ) -> list[dict[str, object]]:
7     # 候选池预评分只用于粗筛和排序，最终策略仍然依赖配额约束下的

```

```

8      # 边际传播收益仿真来决定。
9      degree centrality = nx.degree centrality (graph)
10     betweenness = nx.betweenness centrality (graph)
11     closeness = nx.closeness centrality (graph)
12     core_raw = nx.core_number (graph)
13
14     interaction_norm = normalize ({user: float (values ["interaction_freq"]) for user, values in
15     behavior_map.items ()})
16     activity_norm = normalize ({user: float (values ["activity_count"]) for user, values in behavior_map.
17     items ()})
18     culture_norm = normalize ({user: float (values ["culture_topic"]) for user, values in behavior_map.
19     items ()})
20     core_norm = normalize ({user: float (value) for user, value in core_raw.items ()})
21     degree_norm = normalize ({user: float (graph.degree (user)) for user in graph.nodes})
22
23     bridge_score: dict [str, float] = {}
24     for user in graph.nodes:
25         own_community = community_map.get (user, 0)
26         neighbors = list (graph.neighbors (user))
27         if not neighbors:
28             bridge_score [user] = 0.0
29             continue
30         cross_count = sum (1 for friend in neighbors if community_map.get (friend, 0) != own_community)
31         bridge_score [user] = round (cross_count / len (neighbors), 6)
32
33     bridge_norm = normalize (bridge_score)
34
35     rows: list [dict [str, object]] = []
36     for user in graph.nodes:
37         behavior = behavior_map [user]
38         attr = attr_map [user]
39         row = {
40             "user_id": user,
41             "community_id": community_map.get (user, 0),
42             "degree": graph.degree (user),
43             "degree centrality": round (degree centrality [user], 6),
44             "betweenness": round (betweenness [user], 6),
45             "closeness": round (closeness [user], 6),
46             "core_number": core_raw [user],
47             "core_number_norm": round (core_norm [user], 6),
48             "bridge_score": round (bridge_score [user], 6),
49             "bridge_score_norm": round (bridge_norm [user], 6),
50             "interaction_freq": float (behavior ["interaction_freq"]),
51             "interaction_freq_norm": round (interaction_norm [user], 6),
52             "activity_count": float (behavior ["activity_count"]),
53             "activity_count_norm": round (activity_norm [user], 6),
54             "posts": float (behavior ["posts"]),
55             "culture_topic": float (behavior ["culture_topic"]),
56             "culture_topic_norm": round (culture_norm [user], 6),
57             "active_period": str (behavior ["active_period"]),
58             "grade": attr ["grade"],
59             "major_category": attr ["major_category"],
60             "major": attr ["major"],
61         }
62         row ["is_active_user"] = int (

```



```

60         float(row["activity_count"]) >= ACTIVE_THRESHOLD and float(row["culture_topic"]) >=
CULTURE_THRESHOLD
61     )
62     row["pre_score"] = round(
63         (
64             degree_norm[user]
65             + float(row["betweenness"])
66             + float(row["closeness"])
67             + float(row["core_number_norm"])
68             + float(row["interaction_freq_norm"])
69             + float(row["activity_count_norm"])
70             + float(row["culture_topic_norm"])
71             + float(row["bridge_score_norm"])
72         )
73         / 8.0,
74         6,
75     )
76     rows.append(row)
77
78 rows.sort(key=lambda item: (-float(item["pre_score"]), item["user_id"]))
79 return rows

```

以下代码由 Python 语言编写,实现第 4 问的核心优化过程:先用重复 Monte Carlo 仿真评估传播效果,再在阶段配额约束下通过贪心选人和局部交换完成方案修正。

```

1 def evaluate_schedule(
2     graph: nx.Graph,
3     behavior_map: dict[str, dict[str, float | str]],
4     push_schedule: list[tuple[float, str]],
5     runs: int,
6     base_seed: int,
7 ) -> dict[str, object]:
8     # 所有策略都用重复的 Monte Carlo 仿真评估,保证比较基于期望传播
9     # 效果,而不是单次随机结果。
10    seen_runs: dict[float, list[int]] = {checkpoint: [] for checkpoint in CHECKPOINTS}
11    shared_runs: dict[float, list[int]] = {checkpoint: [] for checkpoint in CHECKPOINTS}
12
13    for run_index in range(runs):
14        rng = random.Random(base_seed + run_index)
15        seen_time, shared_time = simulate_schedule_once(graph, behavior_map, push_schedule, rng)
16        for checkpoint in CHECKPOINTS:
17            seen_value = sum(1 for timestamp in seen_time.values() if timestamp <= checkpoint + 1e-9)
18            shared_value = sum(1 for timestamp in shared_time.values() if timestamp <= checkpoint + 1e
19            -9)
20            seen_runs[checkpoint].append(seen_value)
21            shared_runs[checkpoint].append(shared_value)
22
23    seen_summary = {
24        f"{int(checkpoint)}h": {
25            "mean": round(statistics.mean(values), 4),
26            "std": round(statistics.pstdev(values), 4),
27            "p025": round(empirical_quantile(values, 0.025), 4),
28            "p975": round(empirical_quantile(values, 0.975), 4),
29        }
30    }
31    for checkpoint, values in seen_runs.items()

```

```

30     }
31     shared_summary = {
32         f"{int(checkpoint)}h": {
33             "mean": round(statistics.mean(values), 4),
34             "std": round(statistics.pstdev(values), 4),
35             "p025": round(empirical_quantile(values, 0.025), 4),
36             "p975": round(empirical_quantile(values, 0.975), 4),
37         }
38         for checkpoint, values in shared_runs.items()
39     }
40     return {
41         "schedule": push_schedule,
42         "runs": runs,
43         "seen": seen_summary,
44         "shared": shared_summary,
45     }
46
47
48 def greedy_stage_select(
49     graph: nx.Graph,
50     behavior_map: dict[str, dict[str, float | str]],
51     current_schedule: list[tuple[float, str]],
52     candidate_users: list[str],
53     push_time: float,
54     quota: int,
55     base_seed: int,
56 ) -> tuple[list[tuple[float, str]], list[dict[str, object]]]:
57     # 在当前阶段的名额约束下，按边际收益逐个选人；这是对受限影响力
58     # 最大化问题最直接、也最容易解释的近似方法。
59     schedule = list(current_schedule)
60     selected_users = {user_id for _, user_id in schedule}
61     stage_logs: list[dict[str, object]] = []
62
63     for step in range(1, quota + 1):
64         base_eval = evaluate_schedule(graph, behavior_map, schedule, GREEDY_EVAL_RUNS, base_seed)
65         base_coverage = float(base_eval["seen"]["48h"]["mean"])
66         best_user = None
67         best_schedule = None
68         best_coverage = -1.0
69
70         for user_id in candidate_users:
71             if user_id in selected_users:
72                 continue
73             trial_schedule = schedule + [(push_time, user_id)]
74             trial_eval = evaluate_schedule(graph, behavior_map, trial_schedule, GREEDY_EVAL_RUNS,
75 base_seed)
76             trial_coverage = float(trial_eval["seen"]["48h"]["mean"])
77             if trial_coverage > best_coverage + 1e-9 or (
78                 abs(trial_coverage - best_coverage) <= 1e-9 and (best_user is None or user_id <
79 best_user)
80             ):
81                 best_user = user_id
82                 best_schedule = trial_schedule
83                 best_coverage = trial_coverage
84
85         if best_user is None or best_schedule is None:

```

```

84         break
85
86     stage_logs.append(
87         {
88             "push_time": push_time,
89             "step_in_stage": step,
90             "selected_user": best_user,
91             "coverage_before": round(base_coverage, 4),
92             "coverage_after": round(best_coverage, 4),
93             "marginal_gain": round(best_coverage - base_coverage, 4),
94         }
95     )
96     schedule = best_schedule
97     selected_users.add(best_user)
98
99     return schedule, stage_logs
100
101
102 def local_swap_refine(
103     graph: nx.Graph,
104     behavior_map: dict[str, dict[str, float | str]],
105     initial_schedule: list[tuple[float, str]],
106     candidate_users: list[str],
107     base_seed: int,
108 ) -> tuple[list[tuple[float, str]], list[dict[str, object]]]:
109     # 当贪心结果中两个已选用户覆盖范围高度重叠时，单点替换可以对
110     # 原方案做局部修正。
111     best_schedule = list(initial_schedule)
112     best_summary = evaluate_schedule(graph, behavior_map, best_schedule, LOCAL_SEARCH_RUNS, base_seed)
113     best_objective = objective_tuple(best_summary)
114     logs: list[dict[str, object]] = []
115
116     improved = True
117     while improved:
118         improved = False
119         selected_users = [user_id for _, user_id in best_schedule]
120         unselected_users = [user_id for user_id in candidate_users if user_id not in selected_users]
121
122         for index, (push_time, old_user) in enumerate(best_schedule):
123             for new_user in unselected_users:
124                 trial_schedule = list(best_schedule)
125                 trial_schedule[index] = (push_time, new_user)
126                 trial_summary = evaluate_schedule(
127                     graph,
128                     behavior_map,
129                     trial_schedule,
130                     LOCAL_SEARCH_RUNS,
131                     base_seed + index * 1000,
132                 )
133                 trial_objective = objective_tuple(trial_summary)
134                 if trial_objective > best_objective:
135                     logs.append(
136                         {
137                             "replace_stage_time": push_time,
138                             "old_user": old_user,
139                             "new_user": new_user,

```

```
140         "score_before": round(best_objective[0], 4),
141         "score_after": round(trial_objective[0], 4),
142         "gain": round(trial_objective[0] - best_objective[0], 4),
143     }
144 )
145 best_schedule = trial_schedule
146 best_summary = trial_summary
147 best_objective = trial_objective
148 improved = True
149     break
150 if improved:
151     break
152
153 return best_schedule, logs
```